SISTEMAS DIGITAIS II - 2026/27

TECNOLOGIA SETÚBAL
ESCOLA SUPERIOR POLITÉCNICO SETÚBAL

Laboratório Nº	2
Data	28/10/2026
Docente	José Sousa
Curso	LEEC
Turma	PL1

Alunos		
	202300553	Rodrigo Silva Costa
	2025184378	Diego Alejandro Castro Laya

2.º Trabalho de Laboratório

Gestão dos displays para o jogo Guitar Hero

(Realizações Diretas)

Índice

1 Introdução....2
2 Especificação e arquitetura do sistema....4
2.1 Especificação funcional....4
2.2 Arquitetura a controlar e controlador....4
2.3 Sinais de interface e a sua natureza....5
3 Sincronização das teclas....7
3.1 Sincronizador e anti-ressalto da tecla Start....7
3.2 Sincronizador do Reset....8
4 LFSR de 32 bits....9
5 Memória 4x2....11
6 Temporizador variável....13
7 LED Beat....16
8 Gestão dos displays....17
9 Controlador: máquina de estados por realização direta....20
9.1 Algoritmo e fluxograma....20
9.2 Tabela de estados primitiva (passo 1)....21
9.3 Redução de estados (passos 2 a 4)....22
9.4 Codificação dos estados (passo 5)....23
9.5 Tabela de transições (passo 6)....23
9.6 Saídas nas transições (passo 7)....24
9.7 Equações de excitação e de saída (passos 8 e 9)....24
9.8 Diagrama lógico e código Verilog estrutural (passo 10)....25
9.9 Verificação do controlador (passo 11)....28
10 Integração do sistema....29
10.1 Circuito de topo....29
10.2 Restrições e implementação....31
10.3 Simulação do sistema completo....32
10.4 Integração faseada e ensaio na placa....33
11 Análise crítica do código de fontes externas....35
11.1 LFSR do enunciado....35
11.2 Módulos do 1.º trabalho....35
11.3 Primitivas FDPE e FDCE....35
11.4 IA generativa....35
12 Conclusão....36
Anexo A Testbenches....37

1 Introdução

Este relatório descreve o 2.º Trabalho de Laboratório de Sistemas Digitais II: um sistema para a gestão dos *displays* de 7 segmentos do jogo Guitar Hero na placa Basys3. Um LFSR de 32 *bits* gera uma sequência pseudo-aleatória de cordas e pausas, uma memória de 4x2 *bits* guarda as cordas mostradas nos quatro *displays* e um temporizador variável define o ritmo a que a sequência avança (e acelera) ao longo do "braço da guitarra".

O sistema está dividido em **arquitetura a controlar** e **controlador**. O controlador é uma máquina de estados síncrona por **realização direta** (um *flip-flop* por estado), desenhada a partir do fluxograma do algoritmo segundo a metodologia lecionada e descrita em Verilog no **modelo estrutural**, de forma que cada *flip-flop* corresponde a um estado do fluxograma.

A secção 2 apresenta a especificação, a arquitetura e os sinais de interface; as secções 3 a 8 descrevem os módulos da arquitetura a controlar (código, *testbench* e simulação); a secção 9 descreve o projeto do controlador, pelos 11 passos da metodologia; a secção 10 trata da integração, das restrições, da implementação e do ensaio na placa; a secção 11 analisa o código de fontes externas e a secção 12 conclui. As listagens dos *testbenches* estão no Anexo A.

Ferramentas: Vivado 2026.1 (licença WebPACK) para a FPGA `xc7a35ticpg236-1L` da Basys3 e draw.io para os diagramas. Todos os *testbenches* comparam as saídas com um modelo de referência e terminam com `Simulacao OK` ou `Simulacao FALHOU`; os tempos longos (varrimento, anti-ressalto e temporizador) são parâmetros dos módulos e são reduzidos nas simulações.

2 Especificação e arquitetura do sistema

2.1 Especificação funcional

O sistema tem duas teclas, **Reset** (BTNU) e **Start** (BTNC), os quatro *displays* do jogo e os LEDs de sinalização **Ready** (LD0), **Beat** (LD1) e **Waiting** (LD2). Depois de Reset (secção 2.1 do enunciado):

- inicializa a zeros a memória da sequência (os quatro *displays* ficam apagados) e habilita o LFSR, que passa a avançar livremente;
- acende o LED Ready e espera que se carregue e largue a tecla Start;
- quando se larga Start, apaga o LED Ready e, em ciclo: desloca a memória uma posição, carregando 2 *bits* do LFSR na primeira posição e perdendo a última; troca o estado do LED Beat; inicializa o temporizador variável e espera pelo fim do tempo; acelera ligeiramente o temporizador (parte opcional) e repete.

A tecla Reset tem uma única função: pôr a máquina de estados no estado inicial, de forma **assíncrona** (secção 2.7 do enunciado). Os restantes elementos são inicializados pelas saídas da própria máquina de estados no estado inicial. As únicas exceções são o sincronizador da tecla Start e o varrimento dos *displays* (divisor de *clock* e contador): o primeiro só condiciona uma entrada e o varrimento funciona sempre, arrancando com os valores iniciais da configuração da FPGA; nenhum deles guarda informação do jogo.

O enunciado não define a função do LED Waiting. Como o LED Ready já indica a espera pela tecla Start, o Waiting indica a espera pelo fim do tempo de cada passo (estado S4): fica aceso quase sempre durante o jogo e apaga-se por instantes em cada passo. Para depuração, os LEDs LD10 a LD15 mostram os seis *flip-flops* da máquina de estados (um aceso de cada vez).

2.2 Arquitetura a controlar e controlador

A Figura 1 mostra o diagrama de blocos do sistema e a Tabela 1 a função de cada bloco. As saídas do controlador são os comandos sobre a arquitetura (ClrMem, InitLFSR, EnLFSR, ShiftMem, RstTemp, StartTemp, Faster e TglBeat); as saídas da arquitetura Start e Tempo são as entradas do controlador, fechando a malha de controlo (Fig. 2 do enunciado).

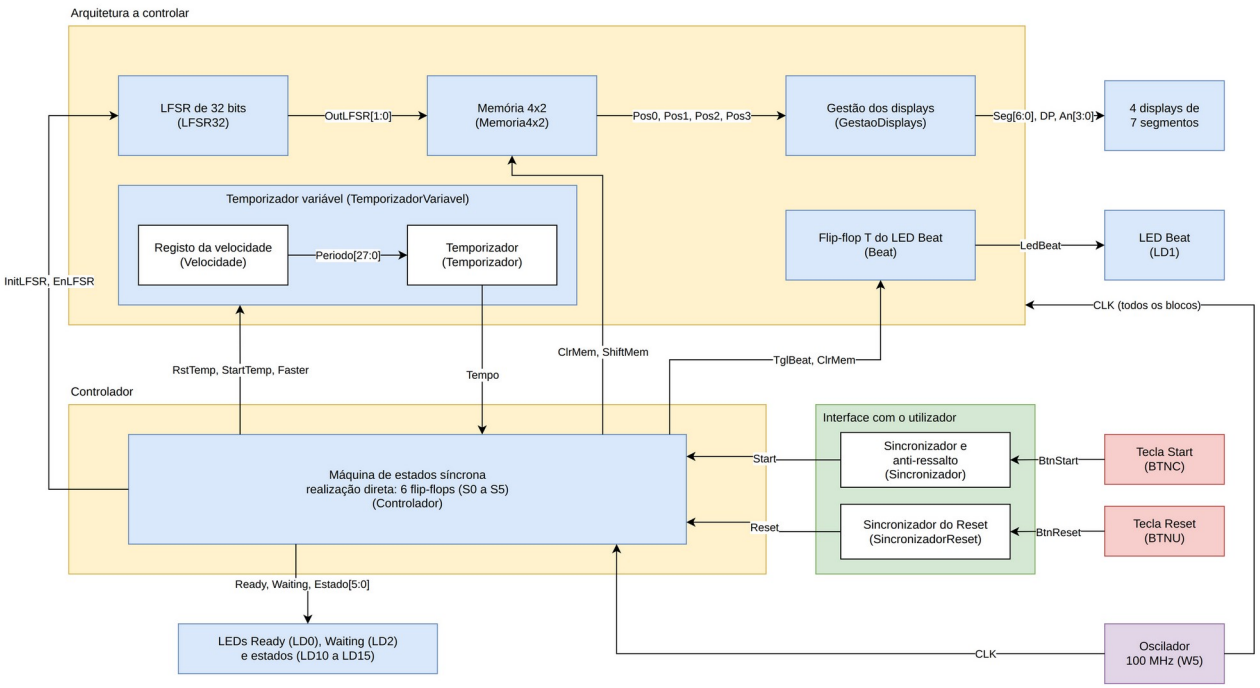


Figura 1: Diagrama de blocos do sistema: arquitetura a controlar, controlador e interface com o utilizador.

Tabela 1: Blocos do sistema, módulos Verilog e origem.

Bloco	Módulo (ficheiro)	Função	Origem
LFSR de 32 *bits*	LFSR32 (lfsr32/lfsr32.v)	Sequência pseudo-aleatória das cordas (2 *bits* por passo)	Enunciado (secção 2.2)
Memória 4x2	Memoria4x2 (memoria4x2/)	Cordas dos 4 *displays*, organizadas em fila	4 × Registo4 do 1.º trabalho
Gestão dos *displays*	GestaoDisplays, TranscodCordas (gestaodisplays/)	Varrimento dos 4 *displays* e transcodificação das cordas	Sistema do 1.º trabalho, transcodificador alterado
Temporizador variável	Velocidade, Temporizador, TemporizadorVariavel (temporizador/)	Tempo de cada passo e aceleração do jogo	Novo
LED Beat	Beat (beat/beat.v)	*Flip-flop* T: o LED troca de estado em cada passo	Novo (primitiva FDCE)
Sincronizador e anti-ressalto	Sincronizador (sincronizador/)	Tecla Start sincronizada com o relógio e sem ressaltos	Novo
Sincronizador do Reset	SincronizadorReset (sincronizador/)	Reset ativado de imediato e libertado em sincronismo com o relógio	Novo (primitivas FDPE)
Controlador	Controlador (controlador/controlador.v)	Máquina de estados por realização direta (6 *flip-flops*)	Novo (FDPE, FDCE e portas lógicas)
Circuito de topo	GuitarHero (guitarhero/)	Liga o controlador à arquitetura e aos pinos da placa	Novo

Para além dos blocos sugeridos no enunciado (LFSR, memória e temporizador), a arquitetura inclui os seguintes:

- **dois sincronizadores**, porque as teclas mudam em qualquer instante: a tecla Start é testada nas decisões da máquina de estados e tem de chegar síncrona e sem ressaltos (secção 3.1); a libertação do Reset tem de ser síncrona para os seis *flip-flops* saírem de S0 no mesmo flanco (secção 3.2);
- um **registo da velocidade**, que guarda o período do temporizador e o encurta em cada passo (parte opcional);
- um ***flip-flop* T** para o LED Beat, que troca de estado com um único comando (TglBeat) por passo.

Os LEDs Ready e Waiting dispensam o *latch* SR sugerido na secção 2.4 do enunciado: as saídas que os comandam estão ativas em todos os estados entre o momento em que o LED acende e aquele em que apaga (Ready em S1 e S2, Waiting em S4). Todos os blocos usam o relógio de 100 MHz da placa.

2.3 Sinais de interface e a sua natureza

A Tabela 2 identifica os sinais entre o controlador, a arquitetura e a placa. Todas as saídas do controlador são de Moore (só dependem do estado) e ativas a *High*.

Tabela 2: Sinais de interface do controlador e da arquitetura a controlar.

Sinal	Bits	Origem → destino	Natureza	Função
CLKBasys	1	Oscilador (W5) → todos os blocos	Relógio, flanco ascendente	Relógio único de 100 MHz
BtnReset	1	Tecla BTNU → Sincronizador do Reset	Assíncrono, ativo a *High*	Inicialização da máquina de estados
BtnStart	1	Tecla BTNC → Sincronizador	Assíncrono, ativo a *High*	Começa o jogo (carregar e largar)
Reset	1	Sincronizador do Reset → Controlador (PRE do FF_S0, CLR dos outros)	Ativação assíncrona, libertação síncrona	Põe a máquina em S0 de imediato
Start	1	Sincronizador → Controlador	Síncrono, ativo a *High*	Tecla Start sem ressaltos (10 ms)
Tempo	1	Temporizador variável → Controlador	Síncrono, ativo a *High*	1 enquanto o temporizador conta
ClrMem	1	Controlador → Memória e Beat (CLR)	Comando assíncrono (CLR)	Limpa a memória e apaga o LED Beat
InitLFSR	1	Controlador → LFSR (Init)	Comando síncrono	Carrega a semente

Sinal	Bits	Origem → destino	Natureza	Função
EnLFSR	1	Controlador → LFSR (Enable)	Comando síncrono	O LFSR avança um passo por ciclo
ShiftMem	1	Controlador → Memória (CE)	Comando síncrono	Desloca a memória e carrega a corda nova
RstTemp	1	Controlador → Temporizador variável	Comando síncrono	Para a contagem e repõe o período inicial
StartTemp	1	Controlador → Temporizador	Comando síncrono	Começa uma contagem com o período atual
Faster	1	Controlador → Registo da velocidade	Comando síncrono	Encurta o período (opcional)
TglBeat	1	Controlador → Beat (CE)	Comando síncrono	Troca o estado do LED Beat
Ready, Waiting	1 + 1	Controlador → LD0, LD2	Sinalização	Espera por Start; espera pelo fim do tempo
Estado[5:0]	6	Controlador → LD10 a LD15	Depuração	Saídas Q dos 6 *flip-flops*
OutLFSR[1:0]	2	LFSR → Memória	Dados, síncrono	Corda nova: 00 pausa, 01 cima, 10 meio, 11 baixo
Pos0 a Pos3	4 × 2	Memória → Gestão dos *displays*	Dados, síncrono	Corda de cada *display* (Pos0 à esquerda)
Periodo[27:0]	28	Registo da velocidade → Temporizador	Dados, síncrono	Tempo de um passo, em ciclos de 10 ns
Seg[6:0], DP	7 + 1	Gestão dos *displays* → cátodos	Ativos a *Low*	Segmentos do *display* ativo; ponto apagado
An[3:0]	4	Gestão dos *displays* → ânodos	Ativos a *Low*	Um *display* de cada vez (2 kHz)
LedBeat	1	Beat → LD1	Sinalização	Pisca ao ritmo do temporizador

3 Sincronização das teclas

3.1 Sincronizador e anti-ressalto da tecla Start

A tecla Start é assíncrona em relação ao relógio e tem ressaltos de alguns milissegundos. Ligada diretamente às portas das decisões de S1 e S2, poderia violar os tempos de preparação de FF_S1 e FF_S2 no mesmo flanco (um podia ver 0 e o outro 1, deixando a máquina com zero ou dois estados ativos), e um ressalto ao carregar faria passar de S2 a S3 antes de a tecla ser largada, começando o jogo cedo de mais.

O `Sincronizador` (Listagem 1) resolve os dois problemas: dois *flip-flops* em série (`meta` e `sinc`, com o atributo `ASYNC_REG` para o Vivado os colocar lado a lado) sincronizam a tecla, e a saída só muda depois de a entrada sincronizada se manter diferente dela durante `ESTAVEL` ciclos seguidos (10 ms). Qualquer ressalto reinicia a contagem.

Listagem 1: Sincronizador e anti-ressalto da tecla Start (`sincronizador/sincronizador.v`)

```
`timescale 1ns / 1ps

// Sincronizador e anti-ressalto da tecla Start.
// 1) Dois flip-flops em série sincronizam a tecla com o relógio de 100 MHz
//    (evitam a metaestabilidade, já que a tecla muda em qualquer instante;
//    ASYNC_REG pede ao Vivado para os colocar lado a lado).
// 2) A saída só muda depois de a entrada sincronizada se manter diferente dela durante
//    ESTAVEL ciclos seguidos; os ressaltos da tecla (mais curtos) são ignorados.
module Sincronizador #(
    parameter ESTAVEL = 1000000     // 10 ms a 100 MHz
    )(
    input Clock,
    input Entrada,
    output reg Saida = 1'b0
    );

     (* ASYNC_REG = "TRUE" *) reg meta = 1'b0, sinc = 1'b0;  // cadeia de sincronização
     reg [31:0] contagem = 0;       // ciclos seguidos com sinc diferente de Saida

     always @(posedge Clock) begin
         meta <= Entrada;
         sinc <= meta;
         if (sinc == Saida)
             contagem <= 0;
         else if (contagem == ESTAVEL - 1) begin
             Saida <= sinc;
             contagem <= 0;
         end
         else
             contagem <= contagem + 1;
     end

endmodule
```

O *testbench* (Listagem 14, no Anexo A) usa `ESTAVEL = 20` e simula ruído curto, carregar e largar a tecla com seis ressaltos e uma falha curta com a tecla carregada, em instantes que não coincidem com o relógio. Verifica que a saída muda uma única vez em cada caso, entre `ESTAVEL` e `ESTAVEL + 3` ciclos depois do último ressalto, e que o ruído é ignorado. Na Figura 2 `contagem` volta a 0 em cada ressalto e `Saida` só muda quando a entrada estabiliza. Resultado: `Simulacao OK (7 verificacoes)`.

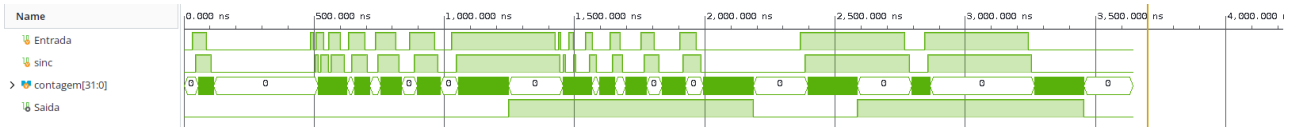


Figura 2: Simulação do sincronizador da tecla Start (ESTAVEL = 20 ciclos).

3.2 Sincronizador do Reset

O Reset tem de pôr a máquina em S0 de imediato, sem esperar pelo relógio (secção 2.7 do enunciado), mas a sua **libertação** não pode ser assíncrona: se a tecla fosse largada junto a um flanco, FF_S0 podia sair do PRE a tempo de carregar D = 0 nesse flanco enquanto FF_S1 ainda estava no CLR. A máquina ficava sem nenhum estado ativo e, como nenhuma equação de excitação tem termos fora dos estados, nunca mais sairia daí. É uma **corrida crítica** entre os sinais assíncronos (passo 5 da metodologia, secção 9.4).

O `SincronizadorReset` (Listagem 2) usa duas primitivas FDPE em série com `PRE` ligado à tecla: ao carregar, a saída vai a 1 de imediato (assíncrono); ao largar, só volta a 0 no 2.º flanco ascendente do relógio, pelo que os seis *flip-flops* saem do *reset* com um ciclo inteiro de folga. Com `INIT = 1`, o Reset também é libertado de forma síncrona depois de configurar a FPGA.

Listagem 2: Sincronizador do Reset (ativação assíncrona, libertação síncrona) (`sincronizador/sincronizadorreset.v`)

```
1  `timescale 1ns / 1ps
2
3  // Sincronizador da tecla Reset: ativação assíncrona, libertação síncrona.
4  // Dois flip-flops FDPE em série com PRE = Entrada:
5  //   - ao carregar Reset, Saida vai a 1 de imediato, sem esperar pelo relógio
6  //     (a máquina de estados é posta em S0 de forma assíncrona, secção 2.7);
7  //   - ao largar Reset, Saida só volta a 0 no 2.º flanco ascendente do relógio.
8  // Sem isto, se a tecla fosse largada mesmo junto a um flanco, FF_S0 podia sair do PRE
9  // (e carregar D = 0) antes de FF_S1 sair do CLR, e a máquina ficava sem nenhum estado ativo.
10 // INIT = 1: depois de configurar a FPGA o Reset também é libertado de forma síncrona.
11 module SincronizadorReset(
12     input Clock,
13     input Entrada,
14     output Saida
15     );
16
17      wire q1;
18
19      (* ASYNC_REG = "TRUE" *) FDPE #(.INIT(1'b1)) ff1
   (.Q(q1),    .C(Clock), .CE(1'b1), .PRE(Entrada), .D(1'b0));
20      (* ASYNC_REG = "TRUE" *) FDPE #(.INIT(1'b1)) ff2
   (.Q(Saida), .C(Clock), .CE(1'b1), .PRE(Entrada), .D(q1));
21
22 endmodule
```

O *testbench* (Listagem 15, no Anexo A) carrega e larga a tecla em dez instantes diferentes do ciclo, incluindo junto ao flanco, e verifica que a saída sobe sem flanco do relógio e desce sempre num flanco ascendente, 2 flancos depois de largar a tecla. Na Figura 3, depois das configurações iniciais (100 ns), `q1` e `Saida` descem em flancos consecutivos; em cada impulso da tecla `Saida` sobe com a entrada. Resultado: `Simulacao OK (34 verificacoes)`.

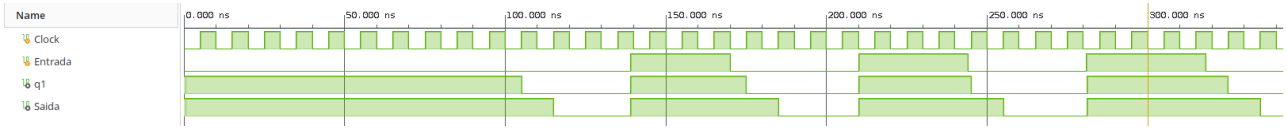


Figura 3: Simulação do sincronizador do Reset: subida imediata e descida síncrona.

4 LFSR de 32 bits

O `LFSR32` (Listagem 3) é o código do enunciado, com comentários. A realimentação é a XNOR das saídas 32, 22, 2 e 1 (*bits* 31, 21, 1 e 0), a combinação da nota de aplicação XAPP052 da Xilinx para 32 *bits*, o que dá o período máximo de $2^{32} - 1$ estados. Com XNOR o estado proibido é tudo a 1 (o LFSR ficaria parado); a semente `32'h03E3180A` é diferente desse valor.

O LFSR é comandado pela máquina de estados: com `Enable` a 1 carrega a semente se `Init` estiver a 1 e avança um passo se `Init` estiver a 0; com `Enable` a 0 mantém o estado. Como `Init` só atua com `Enable` a 1, o estado S0 ativa as duas saídas (InitLFSR e EnLFSR). A saída `OutLFSR = {regLFSR[31], regLFSR[0]}` escolhe a corda (Tabela 3).

Tabela 3: Correspondência entre os 2 bits do LFSR e as cordas.

OutLFSR	Corda	Segmento aceso	Seg[6:0] (a…g)
`00`	Pausa	Nenhum	`1111111`
`01`	Corda de cima	a	`0111111`
`10`	Corda do meio	g	`1111110`
`11`	Corda de baixo	d	`1110111`

O LFSR avança em todos os ciclos do relógio (EnLFSR está ativa em todos os estados), mas a memória só lê um par de *bits* em cada passo do jogo. O número de ciclos que a máquina passa em S1 e S2 depende do tempo que o jogador demora a carregar e largar Start, por isso a sequência de cordas é diferente em cada jogo, apesar de a semente ser fixa.

Listagem 3: LFSR de 32 bits (código do enunciado) (`lfsr32/lfsr32.v`)

```
`timescale 1ns / 1ps

// LFSR de 32 bits que gera a sequência pseudo-aleatória das cordas a tocar.
// Código do enunciado (secção 2.2), com realimentação XNOR nas saídas 32, 22, 2 e 1
// (tabela da nota de aplicação Xilinx XAPP052), o que dá um período de 2^32 - 1 ciclos.
// Com XNOR o estado proibido é 32'hFFFFFFFF (o LFSR ficaria parado), por isso a semente
// tem de ser diferente de tudo a 1; o estado 0 é válido.
// Init e Enable são comandados pela máquina de estados (InitLFSR e EnLFSR):
//   Enable = 1 e Init = 1: carrega a semente;
//   Enable = 1 e Init = 0: avança um passo em cada ciclo de relógio;
//   Enable = 0:            mantém o estado.
// OutLFSR (2 bits) escolhe a corda: 00 pausa, 01 cima, 10 meio, 11 baixo.
module LFSR32 #(
    parameter SEED = 32'h03E3180A   // semente aleatória, diferente de 32'hFFFFFFFF
    )(
    input Clock,
    input Enable,
    input Init,
    output [1:0] OutLFSR
    );

     reg [31:0] regLFSR = 0;
     reg regXNOR = 1;

//   Registo de deslocamento: carrega a semente ou desloca para a esquerda
     always @(posedge Clock)
         if (Enable) begin
             if (Init)
                 regLFSR <= SEED;
             else
                 regLFSR[31:0] <= {regLFSR[30:0], regXNOR};
         end

```

```
34  //   Realimentação: XNOR das saídas 32, 22, 2 e 1 (bits 31, 21, 1 e 0)
35       always @(*)
36           regXNOR = regLFSR[31] ~^ regLFSR[21] ~^ regLFSR[1] ~^ regLFSR[0];
37
38       assign OutLFSR = {regLFSR[31], regLFSR[0]};
39
40  endmodule
```

O *testbench* (Listagem 16, no Anexo A) verifica os casos dirigidos (estado inicial 0, `Init` ignorado com `Enable` a 0, carga da semente, primeiro passo e `Enable` a 0 a manter o estado) e depois compara 100 000 ciclos com um modelo de referência, verificando que o LFSR não passa pelo estado proibido nem repete a semente. Em 100 000 passos saíram 25205 pausas e 24984, 24975 e 24836 notas nas cordas de cima, do meio e de baixo, ou seja, cerca de 25 % cada. Na Figura 4 a semente é carregada aos 25 ns, o primeiro passo dá `07c63015` e o registo fica parado entre 51 e 101 ns, com `Enable` a 0. Resultado: `Simulacao OK (100007 verificacoes)`.

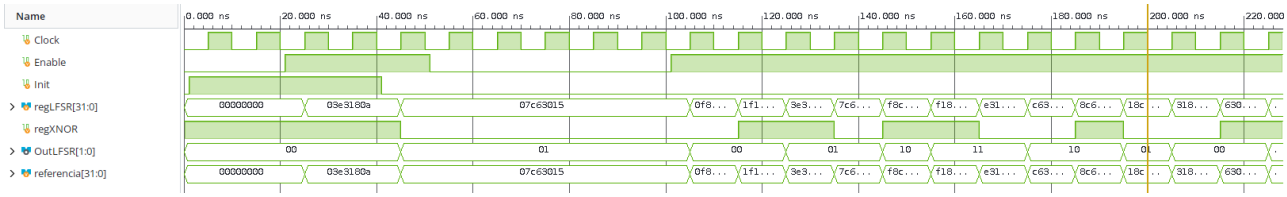


Figura 4: Simulação do LFSR: carga da semente, primeiro passo, Enable = 0 e avanço livre.

5 Memória 4x2

A memória guarda a corda de cada *display* e funciona como uma fila (registo de deslocamento): com `Shift` a 1, no flanco ascendente do relógio, `Pos0` recebe os 2 *bits* do LFSR e cada posição recebe a anterior, perdendo-se a de `Pos3`. `CLR` limpa as quatro posições de forma assíncrona (todas em pausa).

A `Memoria4x2` (Listagem 4) reutiliza quatro `Registo4` do 1.º trabalho (quatro primitivas FDCE cada), em cadeia, dos quais só se usam os 2 *bits* de menor peso: os de maior peso carregam sempre 0 e a síntese elimina-os.

Listagem 4: Memória 4x2 com quatro registos do 1.º trabalho (`memoria4x2/memoria4x2.v`)

```
`timescale 1ns / 1ps

// Memória 4x2 com as cordas dos 4 displays, organizada como uma fila (registo de deslocamento).
// Feita com 4 Registo4 do 1.º trabalho, dos quais só se usam os 2 bits de menor peso
// (os 2 de maior peso carregam sempre 0 e a síntese elimina-os).
// Com Shift a 1, no flanco ascendente do relógio:
//   Pos0 <- Data (2 bits do LFSR), Pos1 <- Pos0, Pos2 <- Pos1, Pos3 <- Pos2
//   (perde-se o valor que estava em Pos3).
// CLR limpa as 4 posições de forma assíncrona (todas as cordas em pausa).
module Memoria4x2(
    input [1:0] Data,
    input Shift,
    input Clk,
    input CLR,
    output [1:0] Pos0,
    output [1:0] Pos1,
    output [1:0] Pos2,
    output [1:0] Pos3
    );

//   Conteúdo dos 4 registos (4 bits cada)
     wire [3:0] Q0, Q1, Q2, Q3;

//   Fila: cada registo carrega o anterior e o primeiro carrega a corda nova
     Registo4 reg0 (.Data({2'b00, Data}), .CE(Shift), .Clk(Clk), .CLR(CLR), .DataOut(Q0));
     Registo4 reg1 (.Data(Q0),            .CE(Shift), .Clk(Clk), .CLR(CLR), .DataOut(Q1));
     Registo4 reg2 (.Data(Q1),            .CE(Shift), .Clk(Clk), .CLR(CLR), .DataOut(Q2));
     Registo4 reg3 (.Data(Q2),            .CE(Shift), .Clk(Clk), .CLR(CLR), .DataOut(Q3));

     assign Pos0 = Q0[1:0];
     assign Pos1 = Q1[1:0];
     assign Pos2 = Q2[1:0];
     assign Pos3 = Q3[1:0];

endmodule
```

O *testbench* (Listagem 17, no Anexo A) compara a fila `{Pos3, Pos2, Pos1, Pos0}` com um modelo de referência sempre que alguma delas muda: primeiro com casos dirigidos (quatro cordas a entrar, `Shift` a 0, *clear* fora do flanco e com prioridade sobre `Shift`) e depois com 2000 ciclos de entradas aleatórias, com *clears* ocasionais. Na Figura 5 as cordas 01, 10, 11, 00 e 10 entram em `Pos0` e avançam uma posição em cada flanco; aos 203 ns o `CLR` apaga tudo sem esperar pelo relógio. Resultado: `Simulacao OK (1013 verificacoes)`.

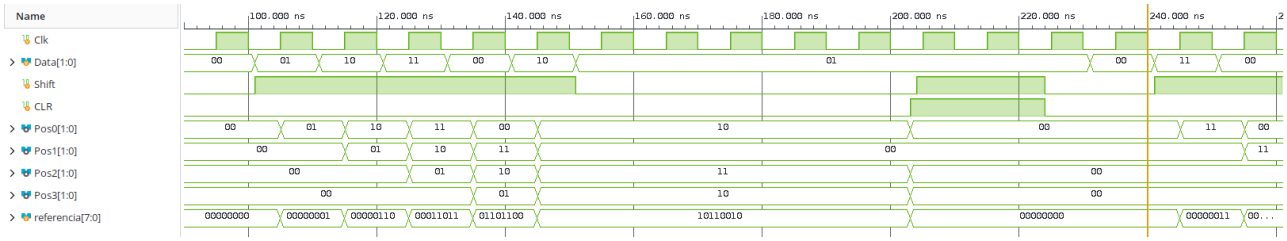


Figura 5: Simulação da memória 4x2: deslocamento das cordas e clear assíncrono.

6 Temporizador variável

O temporizador variável define a duração de cada passo do jogo. Foi dividido em dois módulos, ligados pelo `TemporizadorVariavel` (Listagem 7):

- o `Temporizador` (Listagem 5), pedido na secção 2.3 do enunciado: com `Start` a 1 carrega `Periodo` (o tempo total, em ciclos de 10 ns) e `Tempo` fica a 1 durante `Periodo` ciclos, voltando a 0 sozinho; `Reset` interrompe a contagem;
- o registo da velocidade, `Velocidade` (Listagem 6), que guarda `Periodo`: `Reset` repõe o valor inicial (0,5 s) e cada impulso de `Faster` encurta-o em 5 ms, sem descer abaixo de 0,2 s (parte opcional do enunciado).

Com estes valores o jogo chega à velocidade máxima ao fim de 60 passos (cerca de 21 s). Cada passo dura `Periodo` + 3 ciclos do relógio (estados S3 e S5 e o ciclo de S4 em que Tempo já está a 0), ou seja, mais 30 ns do que o período, o que é desprezável. O contador tem 28 *bits* (até 2,68 s).

Listagem 5: Temporizador (`temporizador/temporizador.v`)

```
`timescale 1ns / 1ps

// Temporizador (secção 2.3 do enunciado).
// Periodo é o tempo total a temporizar, em ciclos do relógio de 100 MHz (10 ns).
// Com Start a 1, no flanco ascendente do relógio, carrega Periodo e começa a contar;
// Tempo fica a 1 durante Periodo ciclos e volta a 0 sozinho no fim da contagem.
// Reset (síncrono) interrompe a contagem e inicializa o temporizador (Tempo = 0).
module Temporizador #(
    parameter N = 28               // bits do contador: até 2^28 ciclos = 2,68 s
    )(
    input Clock,
    input Reset,
    input Start,
    input [N-1:0] Periodo,
    output Tempo
    );

     reg [N-1:0] contagem = 0;      // ciclos que faltam

     always @(posedge Clock)
         if (Reset)
             contagem <= 0;
         else if (Start)
             contagem <= Periodo;
         else if (contagem != 0)
             contagem <= contagem - 1;

     assign Tempo = (contagem != 0);

endmodule
```

Listagem 6: Registo da velocidade do jogo (`temporizador/velocidade.v`)

```
`timescale 1ns / 1ps

// Registo da velocidade do jogo: guarda o período do temporizador (em ciclos de 10 ns).
// Reset (síncrono) repõe o período inicial.
// Com Faster a 1 o período diminui PASSO ciclos no flanco ascendente do relógio,
// sem descer abaixo de MINIMO (parte opcional do enunciado: acelerar ligeiramente).
module Velocidade #(
    parameter N       = 28,
    parameter INICIAL = 50000000,   // 0,5 s por passo da sequência
    parameter PASSO   = 500000,     // acelera 5 ms em cada passo
```

```
    parameter MINIMO  = 20000000    // velocidade máxima: 0,2 s por passo
    )(
    input Clock,
    input Reset,
    input Faster,
    output reg [N-1:0] Periodo = INICIAL
    );

     always @(posedge Clock)
         if (Reset)
             Periodo <= INICIAL;
         else if (Faster && Periodo >= MINIMO + PASSO)
             Periodo <= Periodo - PASSO;

endmodule
```

Listagem 7: Temporizador variável (`temporizador/temporizadorvariavel.v`)

```
`timescale 1ns / 1ps

// Temporizador variável: o registo da velocidade fornece ao temporizador o tempo total a
// temporizar (Periodo). É o bloco "Temporizador variável" da arquitetura a controlar.
//   RstTemp:   inicializa os dois (período inicial e contagem parada);
//   StartTemp: começa uma contagem com o período atual;
//   Faster:    encurta o período para as próximas contagens;
//   Tempo:     1 enquanto a contagem decorre.
module TemporizadorVariavel #(
    parameter N       = 28,
    parameter INICIAL = 50000000,
    parameter PASSO   = 500000,
    parameter MINIMO  = 20000000
    )(
    input Clock,
    input RstTemp,
    input StartTemp,
    input Faster,
    output Tempo
    );

     wire [N-1:0] Periodo;

     Velocidade #(.N(N), .INICIAL(INICIAL), .PASSO(PASSO), .MINIMO(MINIMO)) velocidade (
         .Clock(Clock),
         .Reset(RstTemp),
         .Faster(Faster),
         .Periodo(Periodo)
     );

     Temporizador #(.N(N)) temporizador (
         .Clock(Clock),
         .Reset(RstTemp),
         .Start(StartTemp),
         .Periodo(Periodo),
         .Tempo(Tempo)
     );

endmodule
```

O *testbench* do temporizador (Listagem 18, no Anexo A) usa um contador de 8 *bits* e conta os ciclos em que `Tempo` está a 1 para períodos de 1, 5, 17, 255 e outros; verifica também que `Periodo = 0` não ativa

`Tempo`, que `Reset` interrompe a contagem e tem prioridade sobre `Start` e que um `Start` durante a contagem recomeça com o novo período. Na Figura 6 `contagem` desce de `Periodo` até 0 e `Tempo` fica a 1 durante 1, 5 e 17 ciclos. Resultado: `Simulacao OK (18 verificacoes)`.

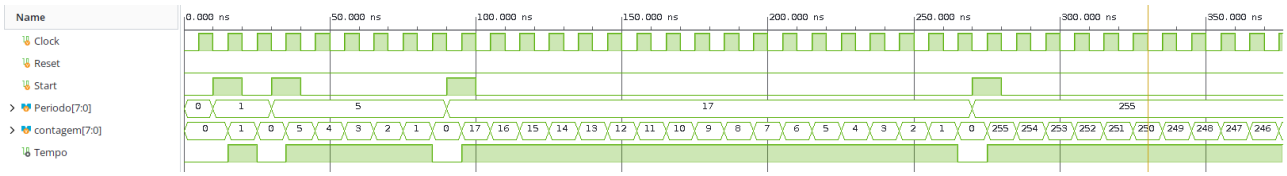


Figura 6: Simulação do temporizador: períodos de 1, 5 e 17 ciclos.

O *testbench* do temporizador variável (Listagem 19, no Anexo A) usa `INICIAL = 40`, `PASSO = 6` e `MINIMO = 20` ciclos e verifica a sequência de períodos 40, 40, 34, 28, 22, 22 e 22 (22 − 6 = 16 seria menor do que o mínimo) e que `RstTemp` a meio da contagem para o temporizador e repõe os 40 ciclos (Figura 7). Resultado: `Simulacao OK (9 verificacoes)`.

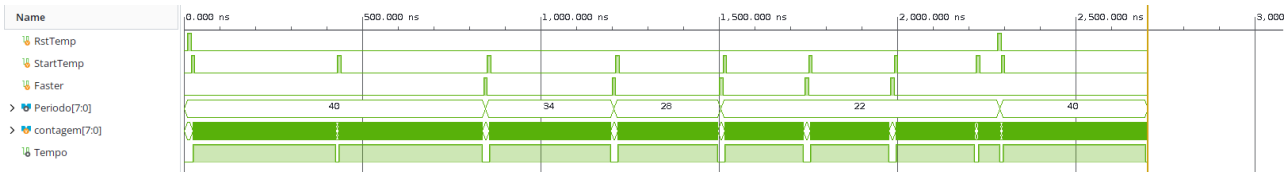


Figura 7: Simulação do temporizador variável: aceleração até ao período mínimo e reposição com RstTemp.

7 LED Beat

O LED Beat pisca ao ritmo do temporizador: o `Beat` (Listagem 8) é um *flip-flop* T feito com uma primitiva FDCE com `D` = /`Q` e `CE` = `Tgl`. Em cada passo do jogo o controlador ativa TglBeat durante um ciclo e o LED troca de estado (aceso num passo, apagado no seguinte). `CLR` está ligado a ClrMem, pelo que o LED apaga com a inicialização. Assim basta um único comando, sem *latch* SR.

Listagem 8: Flip-flop T do LED Beat (`beat/beat.v`)

```
1  `timescale 1ns / 1ps
2
3  // Flip-flop T do LED Beat, feito com uma primitiva FDCE com D = /Q.
4  // Em cada flanco ascendente do relógio com Tgl a 1 o LED troca de estado,
5  // por isso pisca ao ritmo do temporizador variável (Tgl = TglBeat, uma vez por passo).
6  // CLR apaga o LED de forma assíncrona.
7  module Beat(
8      input Clk,
9      input Tgl,
10     input CLR,
11     output Q
12     );
13
14      wire nQ;
15
16      not inv (nQ, Q);
17      FDCE #(.INIT(1'b0)) ff (.Q(Q), .C(Clk), .CE(Tgl), .CLR(CLR), .D(nQ));
18
19 endmodule
```

O *testbench* (Listagem 20, no Anexo A) verifica que `Q` mantém o valor com `Tgl` a 0, troca em cada flanco com `Tgl` a 1 e vai a 0 com `CLR` fora do flanco, mesmo com `Tgl` a 1 (Figura 8). Resultado: `Simulacao OK (11 verificacoes)`.



Figura 8: Simulação do flip-flop T do LED Beat.

8 Gestão dos displays

A gestão dos *displays* (Listagem 10) reutiliza o sistema do 1.º trabalho (secção 2.5 do enunciado): o `DivisorClock` e o `Contador4` fazem o varrimento a 2 kHz (cada *display* aceso 0,5 ms, os quatro atualizados a cada 2 ms), o `multiplexer16x4` escolhe a corda do *display* ativo e o `descodificador2x4`, com os *enables* a 1, ativa um *display* de cada vez. Os registos dos símbolos foram substituídos pela memória 4x2, cujas saídas entram diretamente no *multiplexer*.

O transcodificador foi alterado para as cordas (`TranscodCordas`, Listagem 9 e Tabela 3): cada corda acende um só segmento horizontal (a, g ou d) e a pausa apaga o *display*. A corda nova (`Pos0`) aparece no *display* da esquerda (AN3) e avança para a direita até `Pos3` (AN0). Segmentos e ânodos são ativos a *Low* e o ponto decimal fica apagado.

Listagem 9: Transcodificador das cordas (`gestaodisplays/transcodcordas.v`)

```
`timescale 1ns / 1ps

// Transcodificador das cordas para os segmentos horizontais do display.
// Versão alterada do Transcod7Seg do 1.º trabalho (secção 2.5 do enunciado):
// cada corda acende um só segmento horizontal e a pausa apaga o display.
// Segmentos[6] = a ... Segmentos[0] = g, ativos a Low (0 = segmento aceso).
module TranscodCordas(
    input [1:0] Corda,
    output reg [6:0] Segmentos
    );

     always @(*)
         case (Corda)                //  abcdefg
             2'b00:   Segmentos = 7'b1111111;   // pausa: tudo apagado
             2'b01:   Segmentos = 7'b0111111;   // corda de cima (a)
             2'b10:   Segmentos = 7'b1111110;   // corda do meio (g)
             2'b11:   Segmentos = 7'b1110111;   // corda de baixo (d)
             default: Segmentos = 7'b1111111;   // nunca usado: os 4 casos estão cobertos
         endcase

endmodule
```

Listagem 10: Gestão dos quatro displays (`gestaodisplays/gestaodisplays.v`)

```
`timescale 1ns / 1ps

// Gestão dos 4 displays do jogo, feita com o sistema do 1.º trabalho (secção 2.5):
//   - DivisorClock + Contador4: varrimento dos displays (2 kHz);
//   - multiplexer16x4: corda do display ativo para o transcodificador;
//   - TranscodCordas: transcodificador alterado (um segmento horizontal por corda);
//   - descodificador2x4: ativa um display de cada vez (enables sempre a 1).
// Os registos dos símbolos do 1.º trabalho passam a ser a memória da sequência (Memoria4x2),
// por isso as cordas entram diretamente em Pos0 a Pos3.
// A corda nova (Pos0) aparece no display da esquerda (Anodos[3]) e avança para a direita
// até Pos3, no display da direita (Anodos[0]).
// Segmentos, ponto decimal e ânodos são ativos a Low na Basys3.
module GestaoDisplays #(
    parameter METADE = 25000        // varrimento a 100 MHz / (2 x 25000) = 2 kHz
    )(
    input [1:0] Pos0,
    input [1:0] Pos1,
    input [1:0] Pos2,
    input [1:0] Pos3,
    input Clock,
    output [6:0] Segmentos,
    output DotPoint,
```

```
    output [3:0] Anodos
    );

//   Sinais internos
     wire [3:0] ValorAtivo;                    // corda do display ativo (2 bits de menor peso)
     wire [3:0] DisplayAtivo;                  // saídas do descodificador (ativas a High)
     wire ClockVarrimento;                     // relógio dividido
     wire Control1, Control0;                  // variáveis de varrimento

//   Varrimento: divisor de clock e contador de módulo 4
     DivisorClock #(.METADE(METADE)) divisor (
         .ClockIn(Clock),
         .ClockOut(ClockVarrimento)
     );

     Contador4 contador (
         .Clock(ClockVarrimento),
         .Q1(Control1),
         .Q0(Control0)
     );

//   Multiplexer 16x4: o display i mostra a entrada i (display 3 = esquerda = Pos0)
     multiplexer16x4 mux16x4 (
         .DataIn0({2'b00, Pos3}),
         .DataIn1({2'b00, Pos2}),
         .DataIn2({2'b00, Pos1}),
         .DataIn3({2'b00, Pos0}),
         .DataOut(ValorAtivo),
         .Sel({Control1, Control0})
     );

//   Transcodificador das cordas: segmentos partilhados por todos os displays (ativos a Low)
     TranscodCordas transcod (
         .Corda(ValorAtivo[1:0]),
         .Segmentos(Segmentos)
     );

     assign DotPoint = 1'b1;            // ponto decimal sempre apagado

//   Descodificador 2x4: ativa um display de cada vez
     descodificador2x4 descod (
         .Bit1(Control1),
         .Bit0(Control0),
         .En0(1'b1),
         .En1(1'b1),
         .En2(1'b1),
         .En3(1'b1),
         .Out0(DisplayAtivo[0]),
         .Out1(DisplayAtivo[1]),
         .Out2(DisplayAtivo[2]),
         .Out3(DisplayAtivo[3])
     );

     assign Anodos = ~DisplayAtivo;     // ânodos comuns ligados por transistores PNP: ativos a Low

endmodule
```

O *testbench* (Listagem 21, no Anexo A) usa `METADE = 2` e percorre as 256 combinações das quatro posições; em cada passo do varrimento verifica que há um só ânodo ativo, que os segmentos são os da corda que esse *display* deve mostrar e que os quatro *displays* são varridos. Na Figura 9, com `Pos0` a `Pos3`

= 00, 01, 10 e 11, `Anodos` percorre 1011, 0111, 1110 e 1101 e `Segmentos` mostra, respetivamente, a corda de cima (0111111), a pausa (1111111), a corda de baixo (1110111) e a do meio (1111110). Resultado: `Simulacao OK (2304 verificacoes).`

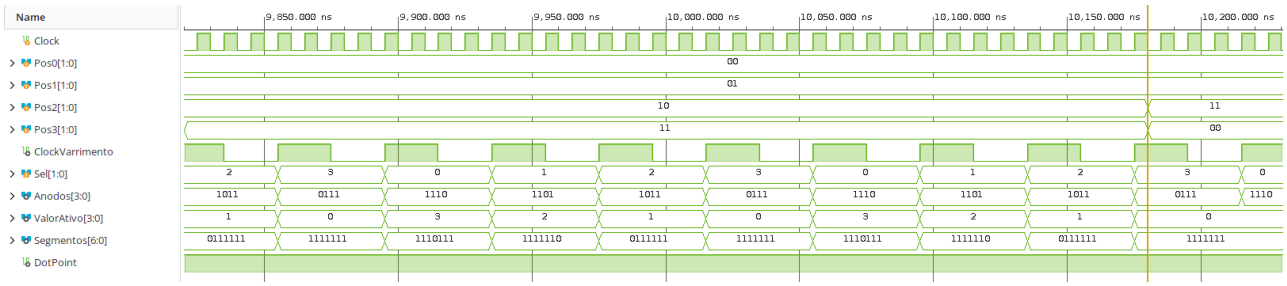


Figura 9: Simulação da gestão dos displays (METADE = 2): varrimento com uma corda diferente em cada posição.

9 Controlador: máquina de estados por realização direta

O controlador foi projetado pela metodologia lecionada, nos 11 passos da Tabela 4. A metodologia foi apresentada para máquinas assíncronas; em cada passo indica-se o que muda numa máquina **síncrona** por realização direta, em que todos os *flip-flops* mudam no mesmo flanco do relógio de 100 MHz.

Tabela 4: Passos do projeto do controlador.

Passo	O que se faz	Secção
1	Construir a tabela de estados primitiva	9.2
2	Eliminar estados redundantes, por inspeção visual e por partições	9.3
3	Agregar estados compatíveis pelo método de fusão	9.3
4	Desenhar a tabela de estados reduzida	9.3
5	Determinar uma codificação dos estados livre de corridas críticas	9.4
6	Desenhar a tabela de transições (excitação)	9.5
7	Determinar o valor das saídas nas transições, sem picos	9.6
8 e 9	Equações de excitação e de saída, sem acidentes	9.7
10	Desenhar o diagrama lógico, com as realimentações	9.8
11	Verificar e testar o circuito	9.9 e 10

9.1 Algoritmo e fluxograma

O fluxograma da Figura 10 foi desenhado a partir dos sinais identificados na secção 2.3: as decisões testam as entradas do controlador (Start e Tempo) e cada estado ativa as saídas indicadas no seu interior (saídas de Moore). O código ao lado de cada estado é o valor de Q5…Q0 (um *flip-flop* por estado).

- **S0** (só com Reset): limpa a memória e o LED Beat, carrega a semente no LFSR e inicializa o temporizador variável.
- **S1**: LED Ready aceso; o LFSR avança livremente; espera que se carregue Start.
- **S2**: Ready continua aceso; espera que se largue Start.
- **S3** (um ciclo): Ready apaga-se; desloca a memória carregando 2 *bits* do LFSR em `Pos0`; troca o LED Beat; começa a contagem do temporizador.
- **S4**: LED Waiting aceso enquanto o temporizador conta (Tempo = 1).
- **S5** (opcional, um ciclo): acelera ligeiramente o temporizador e repete a partir de S3. Sem a parte opcional, o ramo Tempo = 0 iria diretamente para S3.

Cada bloco tem um só estado, no máximo uma decisão e um único estado seguinte para cada combinação das entradas, pelo que o fluxograma é válido. O Reset não aparece como decisão porque atua de forma assíncrona, nos PRE e CLR dos *flip-flops*. Se a tecla Start estiver carregada no momento em que se larga Reset, S1 passa logo a S2 e o jogo começa quando Start for largada: o fluxograma só exige ver a tecla carregada e depois largada.

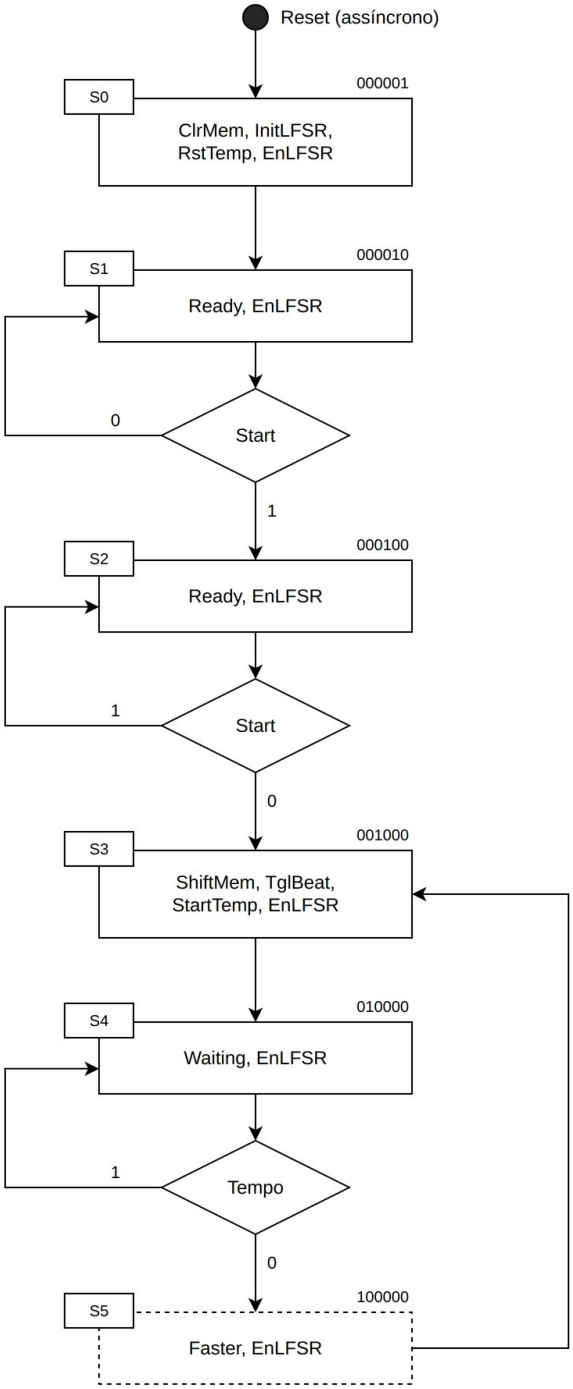


Figura 10: Fluxograma do algoritmo de controlo (S5, a tracejado, é a parte opcional).

9.2 Tabela de estados primitiva (passo 1)

A tabela de estados primitiva (Tabela 5) foi lida diretamente do fluxograma: uma linha por estado e uma coluna por combinação das entradas testadas (Start Tempo, em código de Gray). As células a sombreado indicam que o estado se mantém (estado estável para essa combinação). Start só interessa em S1 e S2 e Tempo só em S4: nas outras linhas o estado seguinte é igual nas quatro colunas. A Figura 11 mostra o diagrama de estados equivalente.

Tabela 5: Tabela de estados primitiva (estado seguinte para Start Tempo = 00, 01, 11 e 10; saídas de Moore).

Estado presente	00	01	11	10	ClrMem	InitLFSR	EnLFSR	ShiftMem	RstTemp	StartTemp	Faster	TglBeat	Ready	Waiting
S0	S1	S1	S1	S1	1	1	1	0	1	0	0	0	0	0
S1	**S1**	**S1**	S2	S2	0	0	1	0	0	0	0	0	1	0
S2	S3	S3	**S2**	**S2**	0	0	1	0	0	0	0	0	1	0
S3	S4	S4	S4	S4	0	0	1	1	0	1	0	1	0	0
S4	S5	**S4**	**S4**	S5	0	0	1	0	0	0	0	0	0	1
S5	S3	S3	S3	S3	0	0	1	0	0	0	1	0	0	0

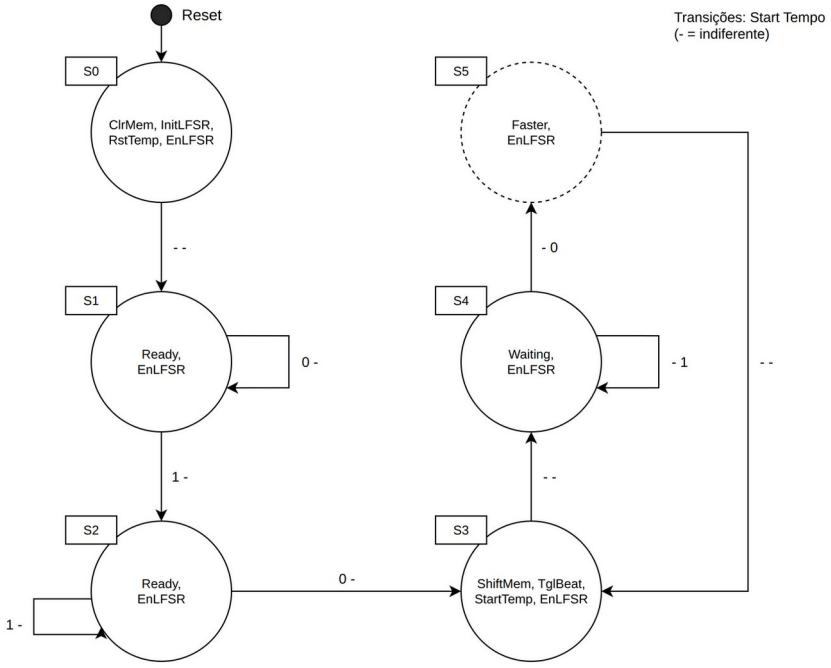


Figura 11: Diagrama de estados (saídas a 1 no interior de cada estado; transições etiquetadas com Start Tempo).

9.3 Redução de estados (passos 2 a 4)

Inspeção visual. Dois estados seriam redundantes se tivessem as mesmas saídas e o mesmo estado seguinte em todas as colunas. Não há nenhum par nessas condições: S1 e S2 têm as mesmas saídas, mas estados seguintes diferentes.

Partições. Começa-se com todos os estados num só bloco, separam-se os que têm saídas diferentes e, depois, os que vão para blocos diferentes com a mesma combinação das entradas, até a partição não mudar (Tabela 6).

Tabela 6: Redução de estados pelo método das partições.

Partição	Justificação
`P0 = (S0 S1 S2 S3 S4 S5)`	Todos os estados num só bloco
`P1 = (S0) (S1 S2) (S3) (S4) (S5)`	Separados pelas saídas: só S1 e S2 têm as mesmas
`P2 = (S0) (S1) (S2) (S3) (S4) (S5)`	Com Start Tempo = 00, S1 vai para S1, no bloco (S1 S2), e S2 vai para S3, no bloco (S3)
`P3 = P2`	A partição não muda: 6 classes, 6 estados

Fusão (passo 3). A tabela está completamente especificada (não há estados seguintes nem saídas indiferentes), pelo que dois estados só são compatíveis se forem equivalentes. O único par com as

mesmas saídas, (S1, S2), já foi separado no passo 2: o grafo de fusão não tem arestas e não há estados a agregar.

Tabela reduzida (passo 4). É a própria tabela primitiva: os seis estados são todos necessários. S1 e S2 esperam, respetivamente, que se carregue e que se largue Start; S3 (deslocar) e S5 (acelerar) duram um só ciclo do relógio; S4 espera pelo fim do tempo; S0 inicializa a arquitetura.

9.4 Codificação dos estados (passo 5)

O enunciado impõe a **realização direta, com um *flip-flop* por estado** (codificação *one-hot*): 6 *flip-flops* em vez dos 3 de uma codificação mínima, mas com uma correspondência unívoca entre cada *flip-flop* e um estado do fluxograma (Tabela 7). FF_S0 é uma primitiva FDPE (com *preset*), porque o Reset tem de o pôr a 1; os restantes são FDCE (com *clear*).

Tabela 7: Codificação dos estados e primitivas usadas.

Estado	Q5 Q4 Q3 Q2 Q1 Q0	Flip-flop	Primitiva	Reset	Valor inicial
S0	`0 0 0 0 0 1`	FF_S0	FDPE	PRE	INIT = 1
S1	`0 0 0 0 1 0`	FF_S1	FDCE	CLR	INIT = 0
S2	`0 0 0 1 0 0`	FF_S2	FDCE	CLR	INIT = 0
S3	`0 0 1 0 0 0`	FF_S3	FDCE	CLR	INIT = 0
S4	`0 1 0 0 0 0`	FF_S4	FDCE	CLR	INIT = 0
S5	`1 0 0 0 0 0`	FF_S5	FDCE	CLR	INIT = 0

Corridas. Em cada transição mudam dois *flip-flops* (o do estado que sai vai a 0 e o do estado que entra vai a 1). Numa máquina assíncrona isto seria uma corrida; aqui todos os *flip-flops* mudam no mesmo flanco do relógio e as entradas D só voltam a ser lidas no flanco seguinte, já estáveis (o Vivado indica uma folga de 5,085 ns em 10 ns). Por isso **não há corridas críticas** na evolução normal da máquina.

O único acontecimento assíncrono é a libertação do Reset, que atua nos PRE e CLR. Se FF_S0 saísse do PRE antes de FF_S1 sair do CLR, a máquina ficava em 000000 sem sair de lá. Esta corrida crítica foi eliminada com o sincronizador do Reset (secção 3.2), que só liberta o Reset num flanco do relógio, com um ciclo inteiro de folga para todos os *flip-flops*.

9.5 Tabela de transições (passo 6)

Com *flip-flops* D a excitação é o próprio estado seguinte (D = Q^+). A Tabela 8 tem uma linha por transição do fluxograma e indica o termo do estado seguinte que lhe corresponde. As outras 58 combinações de Q5…Q0 nunca ocorrem, porque há sempre um só *flip-flop* a 1.

Tabela 8: Tabela de transições (– = indiferente).

Estado presente	Q5…Q0	Start Tempo	Estado seguinte	D5…D0	Termo
S0	`000001`	`– –`	S1	`000010`	S0
S1	`000010`	`0 –`	S1	`000010`	S1·/Start
S1	`000010`	`1 –`	S2	`000100`	S1·Start
S2	`000100`	`1 –`	S2	`000100`	S2·Start
S2	`000100`	`0 –`	S3	`001000`	S2·/Start
S3	`001000`	`– –`	S4	`010000`	S3
S4	`010000`	`– 1`	S4	`010000`	S4·Tempo
S4	`010000`	`– 0`	S5	`100000`	S4·/Tempo
S5	`100000`	`– –`	S3	`001000`	S5

9.6 Saídas nas transições (passo 7)

Sendo de Moore, as saídas só mudam logo a seguir ao flanco do relógio. A Tabela 9 analisa se alguma pode ter um pico numa transição e se isso teria efeito.

Tabela 9: Saídas do controlador nas transições.

Saída	Atua em	Pico numa transição?
ClrMem = S0	CLR **assíncrono** da memória e do Beat	Não: vem diretamente da saída Q de FF_S0
InitLFSR, RstTemp = S0	Entradas síncronas	Não (saída Q de FF_S0)
EnLFSR = S0 + … + S5	Enable síncrono do LFSR	Possível 0 muito curto entre dois flancos, sem efeito: o LFSR só o lê no flanco
Ready = S1 + S2	LED LD0	Possível 0 de nanossegundos de S1 para S2: invisível
ShiftMem, TglBeat, StartTemp = S3	CE e Start síncronos	Não (saída Q de FF_S3)
Waiting = S4, Faster = S5	LED e registo síncrono	Não (saída Q de um só *flip-flop*)

A única saída que atua de forma assíncrona (ClrMem, nos CLR dos registos) vem diretamente de um *flip-flop*, por isso nunca tem picos; as que passam por uma porta OR só vão para entradas síncronas ou para LEDs. Não foi preciso alterar o valor das saídas nas transições.

9.7 Equações de excitação e de saída (passos 8 e 9)

Cada D é a soma dos termos das linhas da tabela de transições que levam a esse estado. Em *one-hot* um único Q a 1 identifica o estado, pelo que cada termo é apenas o estado vezes a condição testada, sem mapas de Karnaugh a minimizar (Tabela 10). Num fluxograma isto lê-se diretamente: as decisões são portas AND (estado · condição) e a reunião dos percursos que entram num estado é uma porta OR.

Tabela 10: Equações de excitação.

Equação	Leitura no fluxograma
`D_S0 = 0`	Nenhum percurso entra em S0 (só a tecla Reset, pelo PRE)
`D_S1 = S0 + S1·/Start`	Vem de S0, ou fica em S1 enquanto Start = 0
`D_S2 = S1·Start + S2·Start`	Vem de S1, ou fica em S2 enquanto Start = 1
`D_S3 = S2·/Start + S5`	Vem de S2 (Start largada) ou de S5
`D_S4 = S3 + S4·Tempo`	Vem de S3, ou fica em S4 enquanto Tempo = 1
`D_S5 = S4·/Tempo`	Vem de S4 quando o tempo acaba (sem S5: D_S3 = S2·/Start + S4·/Tempo)

Sem acidentes. Um pico na lógica de um D não tem efeito, porque o *flip-flop* só lê D no flanco, depois de a lógica estabilizar; além disso Start chega já sincronizada e Tempo é síncrono, pelo que nenhuma entrada assíncrona chega a estas portas. As equações de saída lêem-se nas colunas das saídas da tabela primitiva (soma dos estados em que a saída está a 1):

Tabela 11: Equações de saída.

Saídas	Equação
ClrMem, InitLFSR, RstTemp	`S0`
EnLFSR	`S0 + S1 + S2 + S3 + S4 + S5`
Ready	`S1 + S2`
ShiftMem, TglBeat, StartTemp	`S3`
Waiting	`S4`
Faster	`S5`

9.8 Diagrama lógico e código Verilog estrutural (passo 10)

A Figura 12 é o diagrama lógico do controlador: um *flip-flop* por estado, com a lógica do estado seguinte à esquerda e as saídas de Moore à direita. As **realimentações** (saídas Q dos *flip-flops*, S0 a S5) voltam às portas das decisões pelo barramento desenhado a traço grosso. Os nomes debaixo das portas são os nomes das instâncias no código Verilog.

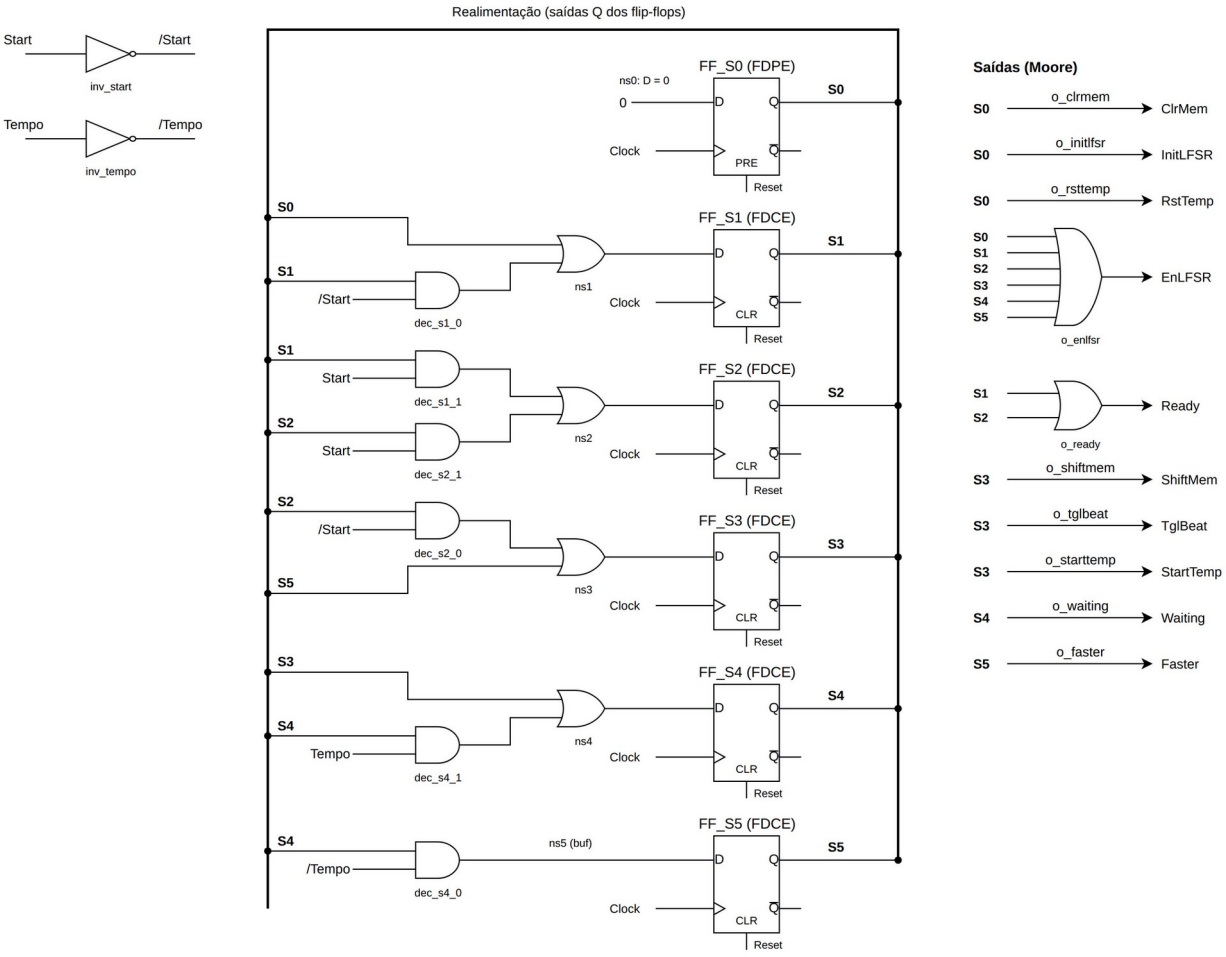


Figura 12: Diagrama lógico do controlador (realização direta), com as realimentações.

O código (Listagem 11) segue o **modelo estrutural** exigido no enunciado: cada estado é uma primitiva (`FF_S0` a `FF_S5`) e cada termo das equações é uma porta lógica primitiva do Verilog (`and`, `or`, `not`, `buf`) com o mesmo nome do diagrama lógico. Assim há uma correspondência unívoca entre cada *flip-flop*, cada estado do fluxograma e cada linha do código. A saída `Estado` liga as saídas Q aos LEDs LD10 a LD15.

Listagem 11: Controlador: máquina de estados por realização direta, no modelo estrutural (`controlador/controlador.v`)

```
1  `timescale 1ns / 1ps
2
3  // Controlador: máquina de estados síncrona por realização direta (um flip-flop por estado),
4  // descrita no modelo estrutural a partir do fluxograma (aula 1):
5  //
6  //   S0 INICIA       ClrMem, InitLFSR, EnLFSR, RstTemp                    -> S1
7  //   S1 ESPERA_START Ready, EnLFSR             Start = 1 -> S2,  senão fica em S1
8  //   S2 LIBERTA      Ready, EnLFSR             Start = 0 -> S3,  senão fica em S2
9  //   S3 DESLOCA      ShiftMem, TglBeat, StartTemp, EnLFSR                 -> S4
10 //   S4 ESPERA_TEMPO Waiting, EnLFSR           Tempo = 0 -> S5,  senão fica em S4
11 //   S5 ACELERA      Faster, EnLFSR            (opcional)                 -> S3
12 //
13 // Flip-flop FF_Sk = estado Sk: só um está a 1 em cada instante (codificação one-hot).
```

```
14 // Reset (tecla) põe a máquina em S0 de forma assíncrona: PRE do FF_S0 e CLR dos restantes.
15 // Os outros elementos da arquitetura são inicializados pelas saídas do estado S0.
16 // As decisões são produtos lógicos (AND) do estado com a condição testada (desmultiplexer)
17 // e a reunião de percursos que entram no mesmo estado é uma soma lógica (OR).
18 // Todas as saídas são de Moore: soma lógica dos estados em que estão ativas.
19 module Controlador(
20     input Clock,
21     input Reset,                    // assíncrono, ativo a High
22     input Start,                    // tecla Start já sincronizada (1 = premida)
23     input Tempo,                    // 1 enquanto o temporizador variável conta
24     output ClrMem,                  // limpa a memória e o LED Beat
25     output InitLFSR,                // carrega a semente no LFSR
26     output EnLFSR,                  // habilita o LFSR (avança livremente)
27     output ShiftMem,                // desloca a memória e carrega a corda nova
28     output RstTemp,                 // inicializa o temporizador variável
29     output StartTemp,               // começa nova contagem do temporizador
30     output Faster,                  // encurta o período do temporizador
31     output TglBeat,                 // troca o estado do LED Beat
32     output Ready,                   // LED Ready
33     output Waiting,                 // LED Waiting
34     output [5:0] Estado             // Q dos 6 flip-flops (LEDs de depuração)
35     );
36
37 //   Estados (saídas Q dos flip-flops) e respetivas entradas D (estado seguinte)
38      wire S0, S1, S2, S3, S4, S5;
39      wire D0, D1, D2, D3, D4, D5;
40
41 //   Condições negadas e percursos de saída das decisões
42      wire nStart, nTempo;
43      wire S1_nStart, S1_Start;      // decisão de S1 (Start = 1 ?)
44      wire S2_Start, S2_nStart;      // decisão de S2 (Start = 0 ?)
45      wire S4_Tempo, S4_nTempo;      // decisão de S4 (Tempo = 1 ?)
46
47 //   ------------------------------------------------------------------ flip-flops (um por estado)
48      FDPE #(.INIT(1'b1)) FF_S0 (.Q(S0), .C(Clock), .CE(1'b1), .PRE(Reset), .D(D0));
49      FDCE #(.INIT(1'b0)) FF_S1 (.Q(S1), .C(Clock), .CE(1'b1), .CLR(Reset), .D(D1));
50      FDCE #(.INIT(1'b0)) FF_S2 (.Q(S2), .C(Clock), .CE(1'b1), .CLR(Reset), .D(D2));
51      FDCE #(.INIT(1'b0)) FF_S3 (.Q(S3), .C(Clock), .CE(1'b1), .CLR(Reset), .D(D3));
52      FDCE #(.INIT(1'b0)) FF_S4 (.Q(S4), .C(Clock), .CE(1'b1), .CLR(Reset), .D(D4));
53      FDCE #(.INIT(1'b0)) FF_S5 (.Q(S5), .C(Clock), .CE(1'b1), .CLR(Reset), .D(D5));
54
55 //   ------------------------------------------------------------------ decisões
56      not inv_start (nStart, Start);
57      not inv_tempo (nTempo, Tempo);
58
59      and dec_s1_0 (S1_nStart, S1, nStart);  // S1 com Start = 0: fica em S1
60      and dec_s1_1 (S1_Start,  S1, Start);   // S1 com Start = 1: vai para S2
61      and dec_s2_1 (S2_Start,  S2, Start);   // S2 com Start = 1: fica em S2
62      and dec_s2_0 (S2_nStart, S2, nStart);  // S2 com Start = 0: vai para S3
63      and dec_s4_1 (S4_Tempo,  S4, Tempo);   // S4 com Tempo = 1: fica em S4
64      and dec_s4_0 (S4_nTempo, S4, nTempo);  // S4 com Tempo = 0: vai para S5
65
66 //   ------------------------------------------------------------------ estado seguinte
67      buf  ns0 (D0, 1'b0);                   // nenhum percurso entra em S0 (só a tecla Reset)
68      or   ns1 (D1, S0, S1_nStart);          // D1 = S0 + S1./Start
69      or   ns2 (D2, S1_Start, S2_Start);     // D2 = S1.Start + S2.Start
70      or   ns3 (D3, S2_nStart, S5);          // D3 = S2./Start + S5
71      or   ns4 (D4, S3, S4_Tempo);           // D4 = S3 + S4.Tempo
72      buf  ns5 (D5, S4_nTempo);              // D5 = S4./Tempo
73
74 //   ------------------------------------------------------------------ saídas (Moore)
75      buf  o_clrmem   (ClrMem,    S0);
```

```
76      buf  o_initlfsr (InitLFSR,  S0);
77      buf  o_rsttemp  (RstTemp,   S0);
78      or   o_enlfsr   (EnLFSR,    S0, S1, S2, S3, S4, S5);
79      or   o_ready    (Ready,     S1, S2);
80      buf  o_shiftmem (ShiftMem,  S3);
81      buf  o_tglbeat  (TglBeat,   S3);
82      buf  o_starttemp(StartTemp, S3);
83      buf  o_waiting  (Waiting,   S4);
84      buf  o_faster   (Faster,    S5);
85
86      assign Estado = {S5, S4, S3, S2, S1, S0};
87
88  endmodule
```

O esquema RTL gerado pelo Vivado a partir do código (RTL Analysis, Open Elaborated Design, Figura 13) mostra a mesma estrutura: FF_S0 como FDPE, FF_S1 a FF_S5 como FDCE e as portas com os nomes do código (`dec_s1_0`, `ns1`, `o_ready`, …). O Vivado apenas decompõe a OR de 6 entradas de EnLFSR em OR de 2 entradas e junta o inversor de Tempo à AND `dec_s4_0`.

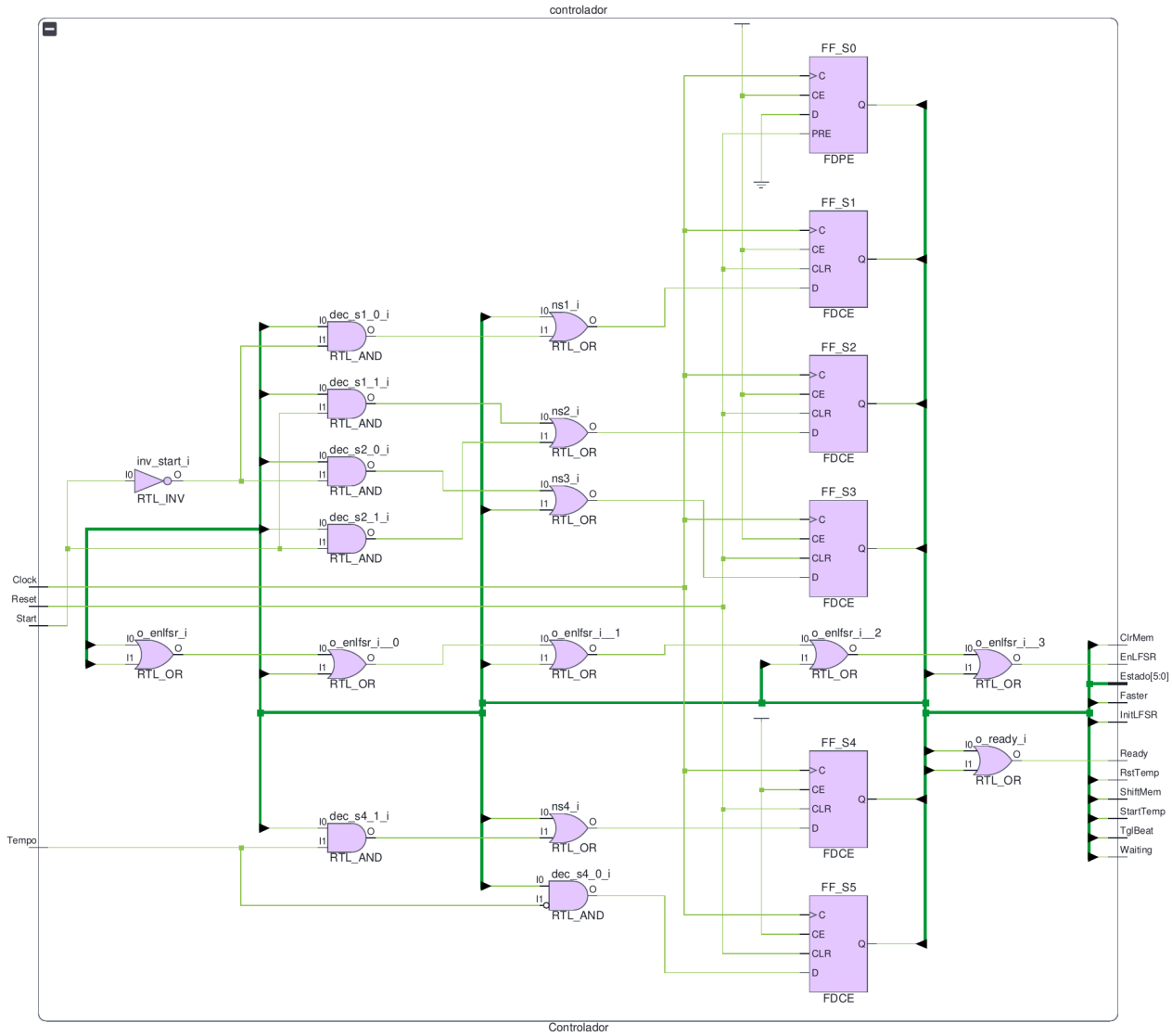


Figura 13: Esquema RTL do controlador gerado pelo Vivado (Elaborated Design).

9.9 Verificação do controlador (passo 11)

O *testbench* do controlador (Listagem 22, no Anexo A) tem um modelo de referência do fluxograma, descrito de forma comportamental com `case`, que corre em paralelo com a máquina de estados. A meio de cada ciclo, e logo depois de cada Reset, compara os seis *flip-flops* e as dez saídas com o modelo e verifica que há sempre um e um só *flip-flop* a 1. A simulação tem duas partes:

1. percurso dirigido por todos os estados e transições do fluxograma, incluindo valores das entradas que não interessam em cada estado (Tempo em S1 e S2, Start em S4) e um Reset a meio do jogo;
2. 5000 ciclos com Start e Tempo aleatórios e Resets assíncronos fora do flanco do relógio.

Os estados foram verificados em 70, 86, 115, 1031, 2769, 1018 ciclos (S0 a S5). Resultado: `Simulacao OK (5089 verificacoes)`. Para confirmar que o *testbench* deteta erros, introduziram-se erros propositados no controlador (uma transição trocada, uma saída no estado errado); a simulação falhou em todos os casos.

Na Figura 14 vê-se o início do percurso dirigido: o Reset aos 107 ns põe S0 a 1 de imediato (com ClrMem, InitLFSR e RstTemp); depois de o largar, a máquina passa a S1 e fica lá enquanto Start = 0, vai para S2 com Start = 1 e para S3 quando Start volta a 0. S3 e S5 duram um ciclo (ShiftMem, StartTemp e TglBeat em S3, Faster em S5), S4 mantém-se enquanto Tempo = 1 e o Reset aos 321 ns volta a pôr a máquina em S0. O modelo de referência (`ref_estado`) coincide sempre com o *flip-flop* ativo.

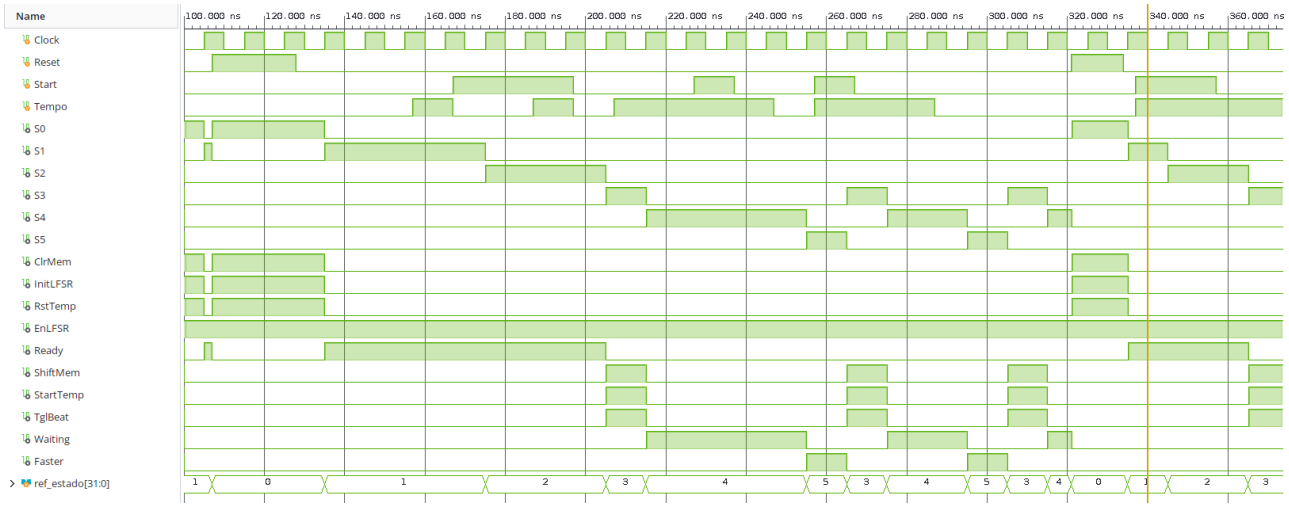


Figura 14: Simulação do controlador: percurso dirigido pelos estados S0 a S5.

10 Integração do sistema

10.1 Circuito de topo

O `GuitarHero` (Listagem 12) liga o controlador à arquitetura a controlar, conforme a Figura 1: as ligações são feitas pelo nome do sinal (ClrMem, ShiftMem, Tempo, …). Os tempos longos são parâmetros (`METADE` do varrimento, `ESTAVEL` do anti-ressalto e `INICIAL`, `PASSO` e `MINIMO` do temporizador), com os valores reais por omissão e valores curtos no *testbench*.

Listagem 12: Circuito de topo: controlador e arquitetura a controlar (`guitarhero/guitarhero.v`)

```
`timescale 1ns / 1ps

// Gestão dos displays para o jogo Guitar Hero (2.º trabalho): circuito de topo.
// Liga o controlador (máquina de estados) à arquitetura a controlar:
//   - SincronizadorReset: Reset ativado de imediato e libertado em sincronismo com o relógio;
//   - Sincronizador: tecla Start sincronizada e sem ressaltos;
//   - LFSR32: sequência pseudo-aleatória das cordas;
//   - Memoria4x2: cordas dos 4 displays (fila);
//   - GestaoDisplays: varrimento dos displays com o transcodificador das cordas;
//   - TemporizadorVariavel: velocidade do jogo;
//   - Beat: flip-flop T do LED Beat.
// As ligações entre o controlador e a arquitetura são feitas pelo nome do sinal.
// Os parâmetros permitem usar tempos curtos na simulação.
module GuitarHero #(
    parameter METADE  = 25000,      // varrimento dos displays a 2 kHz
    parameter ESTAVEL = 1000000,    // anti-ressalto da tecla Start: 10 ms
    parameter INICIAL = 50000000,   // período inicial do temporizador: 0,5 s
    parameter PASSO   = 500000,     // aceleração por passo: 5 ms
    parameter MINIMO  = 20000000    // período mínimo: 0,2 s
    )(
    input CLKBasys,
    input BtnReset,
    input BtnStart,
    output [6:0] Seg,
    output DP,
    output [3:0] An,
    output LedReady,
    output LedBeat,
    output LedWaiting,
    output [5:0] LedEstado
    );

//  Sinais entre o controlador e a arquitetura a controlar
     wire Reset, Start, Tempo;                              // entradas do controlador
     wire ClrMem, InitLFSR, EnLFSR, ShiftMem;               // saídas do controlador
     wire RstTemp, StartTemp, Faster, TglBeat;

//  Sinais internos da arquitetura
     wire [1:0] OutLFSR;
     wire [1:0] Pos0, Pos1, Pos2, Pos3;

//  ------------------------------------------------------------------ controlador
     SincronizadorReset sincronizadorReset (
         .Clock(CLKBasys),
         .Entrada(BtnReset),
         .Saida(Reset)
     );

     Controlador controlador (
         .Clock(CLKBasys),
         .Reset(Reset),
```

```
52          .Start(Start),
53          .Tempo(Tempo),
54          .ClrMem(ClrMem),
55          .InitLFSR(InitLFSR),
56          .EnLFSR(EnLFSR),
57          .ShiftMem(ShiftMem),
58          .RstTemp(RstTemp),
59          .StartTemp(StartTemp),
60          .Faster(Faster),
61          .TglBeat(TglBeat),
62          .Ready(LedReady),
63          .Waiting(LedWaiting),
64          .Estado(LedEstado)
65      );
66
67  //  ------------------------------------------------------------------ arquitetura a controlar
68      Sincronizador #(.ESTAVEL(ESTAVEL)) sincronizador (
69          .Clock(CLKBasys),
70          .Entrada(BtnStart),
71          .Saida(Start)
72      );
73
74      LFSR32 lfsr (
75          .Clock(CLKBasys),
76          .Enable(EnLFSR),
77          .Init(InitLFSR),
78          .OutLFSR(OutLFSR)
79      );
80
81      Memoria4x2 memoria (
82          .Data(OutLFSR),
83          .Shift(ShiftMem),
84          .Clk(CLKBasys),
85          .CLR(ClrMem),
86          .Pos0(Pos0),
87          .Pos1(Pos1),
88          .Pos2(Pos2),
89          .Pos3(Pos3)
90      );
91
92      GestaoDisplays #(.METADE(METADE)) displays (
93          .Pos0(Pos0),
94          .Pos1(Pos1),
95          .Pos2(Pos2),
96          .Pos3(Pos3),
97          .Clock(CLKBasys),
98          .Segmentos(Seg),
99          .DotPoint(DP),
100         .Anodos(An)
101     );
102
103     TemporizadorVariavel #(.INICIAL(INICIAL), .PASSO(PASSO), .MINIMO(MINIMO)) temporizador (
104         .Clock(CLKBasys),
105         .RstTemp(RstTemp),
106         .StartTemp(StartTemp),
107         .Faster(Faster),
108         .Tempo(Tempo)
109     );
110
111     Beat beat (
112         .Clk(CLKBasys),
113         .Tgl(TglBeat),
```

```
114          .CLR(ClrMem),
115          .Q(LedBeat)
116      );
117
118  endmodule
```

10.2 Restrições e implementação

A Tabela 12 mostra a ligação dos sinais à Basys3 (Listagem 13). A tecla Start é a tecla central (BTNC) e o Reset a tecla de cima (BTNU).

Tabela 12: Ligação dos sinais do circuito de topo à Basys3.

Sinal	Pino(s)	Elemento da Basys3
CLKBasys	W5	Oscilador de 100 MHz
BtnReset	T18	Tecla BTNU
BtnStart	U18	Tecla BTNC
LedReady / LedBeat / LedWaiting	U16 / E19 / U19	LEDs LD0 / LD1 / LD2
LedEstado[0] a LedEstado[5]	W3 U3 P3 N3 P1 L1	LEDs LD10 a LD15 (S0 a S5)
Seg[6:0]	W7 W6 U8 V8 U5 V5 U7	Segmentos CA a CG (ativos a *Low*)
DP	V7	Ponto decimal (ativo a *Low*)
An[3:0]	W4 V4 U4 U2	Ânodos AN3 a AN0 (ativos a *Low*)

Como no 1.º trabalho, as restrições definem o relógio de 100 MHz e o relógio do varrimento gerado pelo `DivisorClock` (divisão por 50 000), e o caminho de realimentação do divisor entre os dois relógios é declarado como falso, para evitar o aviso crítico TIMING-8. As teclas (que passam pelos sincronizadores), os LEDs e os *displays* não têm requisitos temporais. A implementação cumpre todas as restrições, com uma folga (WNS) de 5,085 ns, e ocupa 97 LUT e 158 *flip-flops*. O relatório de travessias entre domínios de relógio (`report_cdc`) não tem avisos críticos: os únicos sinais assíncronos, as teclas, chegam aos sincronizadores com `ASYNC_REG`.

Listagem 13: Ficheiro de restrições do circuito de topo (`guitarhero/guitarhero.xdc`)

```
1  ## Restrições do circuito GuitarHero na placa Basys3 (xc7a35ticpg236-1L)
2  ## Tecla Reset = BTNU, tecla Start = BTNC
3  ## LD0 = Ready, LD1 = Beat, LD2 = Waiting, LD10 a LD15 = estados S0 a S5 (um LED por flip-flop)
4
5  ## Relógio de 100 MHz
6  set_property PACKAGE_PIN W5 [get_ports CLKBasys]
7      set_property IOSTANDARD LVCMOS33 [get_ports CLKBasys]
8  create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports CLKBasys]
9
10 ## Relógio do varrimento dos displays (100 MHz / 50000 = 2 kHz), gerado pelo DivisorClock
11 ## (-divide_by = 2 x METADE: atualizar se METADE mudar).
12 create_generated_clock -name clk_varrimento -source [get_ports CLKBasys] -divide_by 50000 [get_pins
   displays/divisor/ClockOut_reg/Q]
13 ## O único caminho clk_varrimento -> sys_clk_pin é a realimentação do próprio divisor (ClockOut ->
   /ClockOut,
14 ## um LUT, ~1,5 ns). Sem este false path o Vivado tenta analisá-lo e, como 50000 períodos não têm período
   comum
15 ## dentro do limite que expande, dá TIMING-8 (Critical Warning); com ele fica só TIMING-9, como no 1.º
   trabalho.
16 set_false_path -from [get_clocks clk_varrimento] -to [get_clocks sys_clk_pin]
17
18 ## Teclas (assíncronas: as duas passam por sincronizadores; o Reset atua de imediato no PRE
19 ## dos flip-flops do SincronizadorReset e a libertação para o controlador já é síncrona)
20 set_false_path -from [get_ports {BtnReset BtnStart}]
21 ## LEDs e displays: sem requisitos de tempo
```

```
set_false_path -to [get_ports {Seg[*] DP An[*] LedReady LedBeat LedWaiting LedEstado[*]}]

## Teclas
set_property PACKAGE_PIN T18 [get_ports BtnReset]
    set_property IOSTANDARD LVCMOS33 [get_ports BtnReset]
set_property PACKAGE_PIN U18 [get_ports BtnStart]
    set_property IOSTANDARD LVCMOS33 [get_ports BtnStart]

## LEDs de sinalização
set_property PACKAGE_PIN U16 [get_ports LedReady]
    set_property IOSTANDARD LVCMOS33 [get_ports LedReady]
set_property PACKAGE_PIN E19 [get_ports LedBeat]
    set_property IOSTANDARD LVCMOS33 [get_ports LedBeat]
set_property PACKAGE_PIN U19 [get_ports LedWaiting]
    set_property IOSTANDARD LVCMOS33 [get_ports LedWaiting]

## LEDs dos estados: LD10 = S0 ... LD15 = S5
set_property PACKAGE_PIN W3 [get_ports {LedEstado[0]}]
set_property PACKAGE_PIN U3 [get_ports {LedEstado[1]}]
set_property PACKAGE_PIN P3 [get_ports {LedEstado[2]}]
set_property PACKAGE_PIN N3 [get_ports {LedEstado[3]}]
set_property PACKAGE_PIN P1 [get_ports {LedEstado[4]}]
set_property PACKAGE_PIN L1 [get_ports {LedEstado[5]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {LedEstado[*]}]

## Displays de 7 segmentos (Seg[6] = a ... Seg[0] = g, ativos a Low)
set_property PACKAGE_PIN W7 [get_ports {Seg[6]}]
set_property PACKAGE_PIN W6 [get_ports {Seg[5]}]
set_property PACKAGE_PIN U8 [get_ports {Seg[4]}]
set_property PACKAGE_PIN V8 [get_ports {Seg[3]}]
set_property PACKAGE_PIN U5 [get_ports {Seg[2]}]
set_property PACKAGE_PIN V5 [get_ports {Seg[1]}]
set_property PACKAGE_PIN U7 [get_ports {Seg[0]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {Seg[*]}]
set_property PACKAGE_PIN V7 [get_ports DP]
    set_property IOSTANDARD LVCMOS33 [get_ports DP]
set_property PACKAGE_PIN U2 [get_ports {An[0]}]
set_property PACKAGE_PIN U4 [get_ports {An[1]}]
set_property PACKAGE_PIN V4 [get_ports {An[2]}]
set_property PACKAGE_PIN W4 [get_ports {An[3]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {An[*]}]

## Configuração
set_property CFGBVS VCCO [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]
```

10.3 Simulação do sistema completo

O *testbench* do sistema (Listagem 23, no Anexo A) usa `METADE = 2`, `ESTAVEL = 4` e um temporizador com `INICIAL = 40`, `PASSO = 4` e `MINIMO = 20` ciclos. É independente da implementação: tem o seu próprio modelo do LFSR e uma fila de referência para a memória, comandados apenas pelos LEDs dos estados (S0 limpa a fila e carrega a semente, S3 desloca). Em cada ciclo verifica que:

- há um e um só estado ativo e os LEDs Ready e Waiting correspondem ao estado;
- a memória é igual à fila de referência e o *display* ativo mostra a corda da posição certa;
- o LED Beat troca de estado uma vez por passo;
- o intervalo entre passos é o período do temporizador mais 3 ciclos (40, 36, 32, … até 20).

O percurso inclui o Reset (ativação imediata e libertação síncrona), a espera em S1, a tecla Start com ressaltos, 12 passos do jogo, um Reset a meio do jogo e um novo jogo, que tem de recomeçar com o período inicial. Resultado: `Simulacao OK (650 verificacoes)`.

Na Figura 15 vê-se o jogo completo: depois do Reset, Ready acende (S1); `Start` (já sem ressaltos) leva a máquina a S2 e, quando é largada, o jogo começa. As cordas entram em `Pos0` e avançam até `Pos3`, `Periodo` diminui de 40 para 20 ciclos e o LED Beat troca a cada passo. O Reset a meio do jogo limpa a memória e o LED Beat e repõe o período de 40 ciclos.

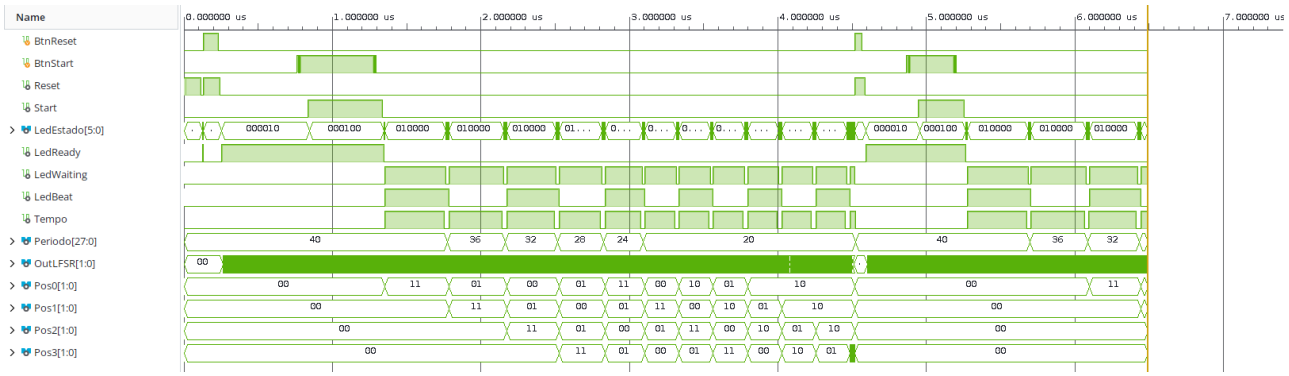


Figura 15: Simulação do sistema completo: Reset, Start, 12 passos do jogo, Reset a meio do jogo e novo jogo.

A Figura 16 mostra o primeiro passo em pormenor. Quando `Start` volta a 0, a máquina passa de S2 a S3: nesse ciclo ShiftMem carrega a corda do LFSR (11) em `Pos0` e StartTemp inicia o temporizador, que mantém `Tempo` a 1 durante 40 ciclos (S4). Quando `Tempo` volta a 0, a máquina passa por S5 (Faster: o período passa a 36) e regressa a S3, onde a corda seguinte (01) entra em `Pos0` e o 11 passa para `Pos1`. Entre os dois passos decorrem 43 ciclos (40 + 3).

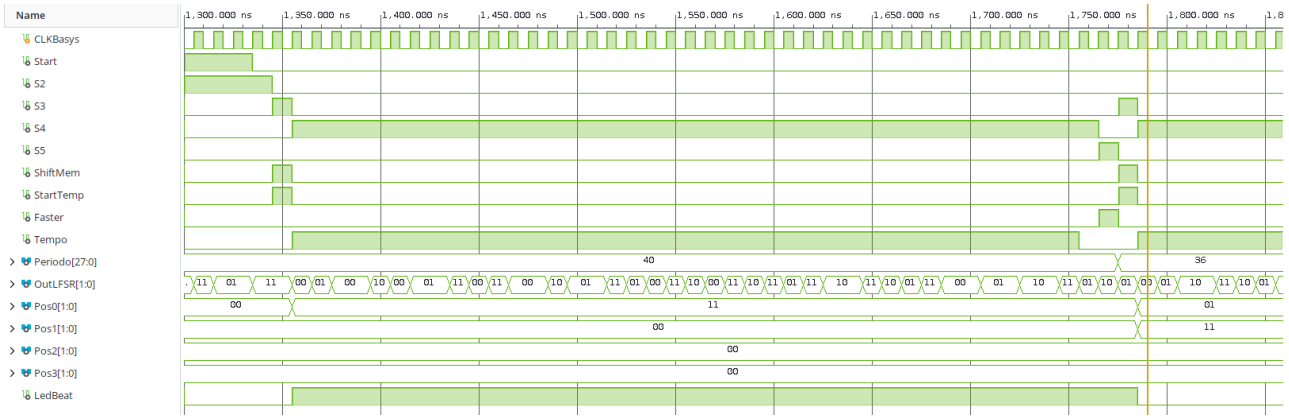


Figura 16: Pormenor da simulação do sistema: dois passos do jogo (S3, S4, S5 e S3).

10.4 Integração faseada e ensaio na placa

A integração foi feita módulo a módulo, como pede o enunciado: cada módulo foi simulado isoladamente (secções 3 a 8), depois o controlador sozinho (secção 9.9) e, por fim, o sistema completo (secção 10.3). Na placa, o ensaio segue a ordem da Tabela 13; os LEDs LD10 a LD15 mostram em cada instante o *flip-flop* ativo, o que permite seguir o fluxograma na própria placa.

Tabela 13: Procedimento de ensaio na Basys3 e resultado esperado.

Passo	Ação	Resultado esperado
1	Programar a FPGA com guitarhero.bit	Displays apagados; LD11 (S1) e LD0 (Ready) acesos
2	Carregar BTNC (Start) e mantê-la carregada	LD12 (S2) aceso; Ready continua aceso; displays apagados
3	Largar BTNC	Ready apaga; LD14 (S4) e LD2 (Waiting) acesos; as cordas entram pelo *display* da esquerda e avançam para a direita a cada 0,5 s; LD1 (Beat) troca a cada passo
4	Esperar cerca de 20 s	A sequência acelera até 0,2 s por passo e mantém essa velocidade
5	Carregar BTNU (Reset) a meio do jogo	Displays e LD1 apagam de imediato; LD10 (S0) aceso enquanto a tecla está carregada
6	Largar BTNU e repetir os passos 2 e 3	Novo jogo, outra vez a 0,5 s por passo e com outra sequência de cordas

[A COMPLETAR NO LABORATÓRIO: confirmar o resultado de cada passo da Tabela 13 e substituir as fotografias abaixo]

[A INSERIR NO LABORATÓRIO: fotografia da placa depois do Reset (passo 1)]

Figura 17: Ensaio na placa Basys3 depois do Reset: LED Ready e LD11 (S1) acesos, displays apagados.

[A INSERIR NO LABORATÓRIO: fotografia da placa durante o jogo (passo 3)]

Figura 18: Ensaio na placa Basys3 durante o jogo: cordas nos quatro displays, LEDs Waiting e LD14 (S4) acesos.

11 Análise crítica do código de fontes externas

11.1 LFSR do enunciado

O `LFSR32` do enunciado funciona e as derivações estão certas (32, 22, 2 e 1 na XAPP052 para 32 *bits*), mas tem alguns pormenores a notar:

- o comentário pede uma semente "<> 0", o que é válido para um LFSR com XOR; com XNOR o estado proibido é tudo a 1 e o 0 é um estado válido (é, aliás, o valor do registo depois de configurar a FPGA). A semente `32'h03E3180A` serve nos dois casos;
- `Init` só atua com `Enable` a 1, porque está dentro do `if (Enable)`: o estado inicial da máquina tem de ativar InitLFSR e EnLFSR ao mesmo tempo;
- o valor inicial de `regXNOR` (`reg regXNOR = 1`) não tem efeito, porque é um sinal combinatório atribuído num `always @(*)`;
- `OutLFSR` usa os *bits* 31 e 0, que em ciclos consecutivos estão correlacionados (o LFSR desloca um *bit* por ciclo); como a memória só lê um par de *bits* a cada `Periodo` + 3 ciclos, os pares lidos são praticamente independentes, o que a simulação confirmou (cerca de 25 % para cada corda).

O código foi usado sem alterações, só com comentários.

11.2 Módulos do 1.º trabalho

O `Registo4`, o `DivisorClock`, o `Contador4`, o `multiplexer16x4` e o `descodificador2x4` foram reutilizados sem alterações: os projetos e os *scripts* do 2.º trabalho referem os ficheiros da pasta do 1.º trabalho, sem os copiar, pelo que uma correção feita num módulo passa a valer nos dois trabalhos. O transcodificador foi substituído por uma versão nova (`TranscodCordas`), porque a função é diferente (um segmento horizontal por corda).

11.3 Primitivas FDPE e FDCE

As primitivas seguem os modelos do Vivado: `PRE` e `CLR` são assíncronos e ativos a *High*, `CE` é ativo a *High* e `INIT` define o valor depois de configurar a FPGA. FF_S0 tem `INIT = 1` e os outros `INIT = 0`, pelo que a máquina arranca em S0 mesmo sem carregar Reset; o sincronizador do Reset também tem `INIT = 1`, mantendo o Reset ativo durante os dois primeiros ciclos depois da configuração.

11.4 IA generativa

Foi usada IA generativa como apoio na escrita dos *testbenches* e dos modelos de referência, na revisão do código Verilog e dos comentários, na preparação dos diagramas no draw.io e na correção da redação deste relatório, bem como para confirmar que o relatório inclui os elementos pedidos no enunciado. Todo o código e todos os diagramas foram revistos pelo grupo e as simulações foram corridas e analisadas pelo grupo.

12 Conclusão

Desenvolveu-se o sistema de gestão dos *displays* do jogo Guitar Hero: um LFSR de 32 *bits* gera a sequência de cordas e pausas, uma memória 4x2 guarda o que cada *display* mostra e um temporizador variável define e acelera o ritmo do jogo. A gestão dos *displays* reutilizou o sistema do 1.º trabalho, com um novo transcodificador.

O controlador, o cerne do trabalho, foi desenhado a partir do fluxograma pelos 11 passos da metodologia e realizado de forma direta, com um *flip-flop* por estado, no modelo estrutural: cada *flip-flop* corresponde a um estado do fluxograma e cada porta a um termo das equações, o que o esquema RTL do Vivado confirmou. O trabalho mostrou também a importância da sincronização das entradas assíncronas: a tecla Start precisa de sincronizador e anti-ressalto, e a libertação do Reset tem de ser síncrona para evitar uma corrida crítica que deixaria a máquina sem estado ativo.

Os 10 *testbenches* (um por módulo, o do controlador e o do sistema completo) comparam as saídas com modelos de referência e terminam todos com `Simulacao OK`; a implementação cumpre as restrições temporais.

[A COMPLETAR NO LABORATÓRIO: acrescentar o resultado do ensaio na placa Basys3 (secção 10.4)]

Anexo A Testbenches

Listagens dos *testbenches* das secções 3 a 10. Todos verificam as saídas automaticamente e terminam com `Simulacao OK` ou `Simulacao FALHOU`; as vistas das formas de onda usadas nas figuras estão nos ficheiros `.wcfg` ao lado de cada *testbench*.

Listagem 14: Testbench do sincronizador da tecla Start (`sincronizador/sincronizador_teste1.v`)

```
`timescale 1ns / 1ps

// Testbench do sincronizador / anti-ressalto (ESTAVEL = 20 ciclos para a simulação).
// Simula carregar e largar a tecla com ressaltos em instantes que não coincidem com o
// relógio e verifica que a saída muda uma só vez em cada caso, ESTAVEL + 2 ciclos depois
// do último ressalto, e que impulsos curtos (ruído) são ignorados.
module sincronizador_teste1();

//   Declaração das entradas
     reg Clock = 0;
     reg Entrada = 0;

//   Declaração da saída
     wire Saida;

     localparam ESTAVEL = 20;

//   Instanciação da UUT e ligação das entradas e saídas
     Sincronizador #(.ESTAVEL(ESTAVEL)) UUT (
         .Clock(Clock),
         .Entrada(Entrada),
         .Saida(Saida)
     );

//   Relógio de 100 MHz (período de 10ns), como o da Basys3
     always #5 Clock = ~Clock;

//   Conta as mudanças da saída e guarda o instante da última
//   (em t = 0 a passagem de x para o valor inicial não conta)
     integer subidas = 0, descidas = 0;
     time t_mudanca = 0;
     always @(posedge Saida) if ($time > 0) begin subidas = subidas + 1; t_mudanca = $time; end
     always @(negedge Saida) if ($time > 0) begin descidas = descidas + 1; t_mudanca = $time; end

     integer erros = 0, verificacoes = 0;
     task verifica(input integer s, input integer d, input [8*48:1] caso);
         begin
             verificacoes = verificacoes + 1;
             if (subidas != s || descidas != d) begin
                 $display("ERRO t=%0t (%0s): %0d subidas e %0d descidas (esperado %0d e %0d)",
                          $time, caso, subidas, descidas, s, d);
                 erros = erros + 1;
             end
         end
     endtask

//   Tecla com ressaltos: muda de valor várias vezes (intervalos < ESTAVEL ciclos) e fica em v
     integer k;
     time t_fim;
     task ressalta(input v);
         begin
             for (k = 0; k < 6; k = k + 1) begin
                 Entrada = v;  #(13 + 17 * k);
                 Entrada = ~v; #(7 + 11 * k);
```

```
            end
            Entrada = v;
            t_fim = $time;
        end
    endtask

    initial begin
        #33;
        // Ruído curto (5 ciclos): ignorado
        Entrada = 1; #53 Entrada = 0;
        #400 verifica(0, 0, "impulso curto ignorado");

        // Carregar a tecla com ressaltos
        ressalta(1'b1);
        #400 verifica(1, 0, "carregar: uma so subida");
        verificacoes = verificacoes + 1;
        if (t_mudanca - t_fim < ESTAVEL * 10 || t_mudanca - t_fim > (ESTAVEL + 3) * 10) begin
            $display("ERRO: a saida subiu %0d ns depois do ultimo ressalto (esperado ~%0d ns)",
                     t_mudanca - t_fim, (ESTAVEL + 2) * 10);
            erros = erros + 1;
        end

        // Largar a tecla com ressaltos
        ressalta(1'b0);
        #400 verifica(1, 1, "largar: uma so descida");

        // Impulso a 0 curto com a tecla carregada: ignorado
        Entrada = 1; #400;
        verifica(2, 1, "tecla carregada outra vez");
        Entrada = 0; #77 Entrada = 1;
        #400 verifica(2, 1, "falha curta com a tecla carregada ignorada");
        Entrada = 0; #400 verifica(2, 2, "tecla largada");

        if (erros == 0) $display("Simulacao OK (%0d verificacoes)", verificacoes);
        else            $display("Simulacao FALHOU (%0d erros em %0d verificacoes)", erros, verificacoes);
        $finish;
    end

endmodule
```

Listagem 15: Testbench do sincronizador do Reset (`sincronizador/sincronizadorreset_teste1.v`)

```
`timescale 1ns / 1ps

// Testbench do sincronizador do Reset.
// Verifica que a saída sobe de imediato (assíncrona) ao carregar a tecla, em qualquer
// instante do ciclo, e que só desce num flanco ascendente do relógio, 2 flancos depois de
// a tecla ser largada (também depois das configurações iniciais da FPGA).
module sincronizadorreset_teste1();

//   Declaração das entradas
     reg Clock = 0;
     reg Entrada = 0;

//   Declaração da saída
     wire Saida;

//   Instanciação da UUT e ligação das entradas e saídas
     SincronizadorReset UUT (
         .Clock(Clock),
         .Entrada(Entrada),
```

```
20          .Saida(Saida)
21      );
22
23  //  Relógio de 100 MHz (período de 10ns), como o da Basys3
24      always #5 Clock = ~Clock;
25
26      integer erros = 0, verificacoes = 0;
27      task verifica(input esperado, input [8*48:1] caso);
28          begin
29              verificacoes = verificacoes + 1;
30              if (Saida !== esperado) begin
31                  $display("ERRO t=%0t (%0s): Saida=%b (esperado %b)", $time, caso, Saida, esperado);
32                  erros = erros + 1;
33              end
34          end
35      endtask
36
37  //  A saída só pode descer num flanco ascendente do relógio
38      always @(negedge Saida)
39          if ($time > 0 && ($time % 10) != 5) begin
40              $display("ERRO t=%0t: Saida desceu fora do flanco do relogio", $time);
41              erros = erros + 1;
42          end
43
44  //  Conta os flancos do relógio desde que a tecla foi largada até a saída descer
45      integer flancos;
46      task larga_e_conta;
47          begin
48              Entrada = 0;
49              flancos = 0;
50              while (Saida === 1'b1 && flancos < 10) @(posedge Clock) begin
51                  #1 flancos = flancos + 1;
52              end
53              verificacoes = verificacoes + 1;
54              if (flancos != 2) begin
55                  $display("ERRO t=%0t: a saida desceu ao fim de %0d flancos (esperado 2)", $time, flancos);
56                  erros = erros + 1;
57              end
58          end
59      endtask
60
61      integer k;
62      initial begin
63          #1   verifica(1'b1, "ativo depois de configurar a FPGA");
64          #99;                                            // fim das configurações iniciais (GSR)
65          flancos = 0;
66          while (Saida === 1'b1 && flancos < 10) @(posedge Clock) begin
67              #1 flancos = flancos + 1;
68          end
69          verificacoes = verificacoes + 1;
70          if (flancos != 2) begin
71              $display("ERRO: depois do GSR a saida desceu ao fim de %0d flancos (esperado 2)", flancos);
72              erros = erros + 1;
73          end
74
75          // Carregar e largar a tecla em vários instantes do ciclo (incluindo junto ao flanco)
76          for (k = 0; k < 10; k = k + 1) begin
77              #(23 + k);
78              Entrada = 1;
79              #0.1 verifica(1'b1, "sobe de imediato, sem flanco");
80              #(31 + 3 * k);
81              verifica(1'b1, "ativo enquanto a tecla esta carregada");
```

```
            larga_e_conta;
        end

        // Impulso muito curto entre dois flancos: também é aceite
        @(negedge Clock) Entrada = 1;
        #1 Entrada = 0;
        verifica(1'b1, "impulso curto ativa o Reset");
        larga_e_conta;

        if (erros == 0) $display("Simulacao OK (%0d verificacoes)", verificacoes);
        else            $display("Simulacao FALHOU (%0d erros em %0d verificacoes)", erros, verificacoes);
        $finish;
    end

endmodule
```

Listagem 16: Testbench do LFSR de 32 bits (`lfsr32/lfsr32_teste1.v`)

```
`timescale 1ns / 1ps

// Testbench do LFSR de 32 bits.
// Parte 1: casos dirigidos (estado inicial, carga da semente, Enable = 0 mantém o estado).
// Parte 2: 100000 ciclos comparados com um modelo de referência; verifica-se também que o
//          LFSR nunca fica parado e que as 4 cordas aparecem com frequências parecidas.
module lfsr32_teste1();

//   Declaração das entradas
    reg Clock = 0;
    reg Enable = 0, Init = 0;

//   Declaração da saída
    wire [1:0] OutLFSR;

    localparam [31:0] SEMENTE = 32'h03E3180A;

//   Instanciação da UUT e ligação das entradas e saídas
    LFSR32 #(.SEED(SEMENTE)) UUT (
        .Clock(Clock),
        .Enable(Enable),
        .Init(Init),
        .OutLFSR(OutLFSR)
    );

//   Relógio de 100 MHz (período de 10ns), como o da Basys3
    always #5 Clock = ~Clock;

//   Modelo de referência: realimentação = XNOR dos bits 31, 21, 1 e 0
    reg [31:0] referencia = 0;
    always @(posedge Clock)
        if (Enable)
            referencia <= Init ? SEMENTE
                               : {referencia[30:0], ~(referencia[31] ^ referencia[21] ^ referencia[1] ^
referencia[0])};

    integer erros = 0, verificacoes = 0;
    task verifica(input [31:0] esperado, input [8*40:1] caso);
        begin
            verificacoes = verificacoes + 1;
            if (UUT.regLFSR !== esperado || OutLFSR !== {esperado[31], esperado[0]}) begin
                if (erros < 10) $display("ERRO t=%0t (%0s): regLFSR=%h OutLFSR=%b (esperado %h)",
                                         $time, caso, UUT.regLFSR, OutLFSR, esperado);
```

```
                erros = erros + 1;
            end
        end
    endtask

    integer n;
    integer conta [0:3];          // quantas vezes saiu cada corda
    reg [31:0] guardado;
    initial begin
        conta[0] = 0; conta[1] = 0; conta[2] = 0; conta[3] = 0;
        // --- Parte 1: casos dirigidos ------------------------------------------
        #1   verifica(32'h0, "estado inicial = 0");
        // As entradas mudam a meio do período (relógio sobe em 5, 15, 25, ...)
        Init = 1;          #20 verifica(32'h0, "Enable=0 ignora Init");
        Enable = 1;        #10 verifica(SEMENTE, "Enable=1, Init=1 carrega semente");
                           #10 verifica(SEMENTE, "Init mantido: continua na semente");
        Init = 0;          #10 verifica({SEMENTE[30:0], ~(SEMENTE[31]^SEMENTE[21]^SEMENTE[1]^SEMENTE[0])},
"primeiro passo");
        Enable = 0;        guardado = UUT.regLFSR;
                           #50 verifica(guardado, "Enable=0 mantem durante 5 ciclos");
        Enable = 1;        #10 verifica(referencia, "volta a avancar");

        // --- Parte 2: 100000 ciclos contra o modelo ----------------------------
        for (n = 0; n < 100000; n = n + 1) begin
            #10 verifica(referencia, "comparacao com o modelo");
            conta[OutLFSR] = conta[OutLFSR] + 1;
            if (UUT.regLFSR == 32'hFFFFFFFF) begin
                $display("ERRO t=%0t: LFSR no estado proibido (tudo a 1)", $time);
                erros = erros + 1;
            end
            if (UUT.regLFSR == SEMENTE) begin
                $display("ERRO t=%0t: a sequencia repetiu-se ao fim de %0d passos", $time, n);
                erros = erros + 1;
            end
        end
        $display("Cordas em 100000 passos: pausa %0d, cima %0d, meio %0d, baixo %0d",
                 conta[0], conta[1], conta[2], conta[3]);
        for (n = 0; n < 4; n = n + 1)
            if (conta[n] < 23000 || conta[n] > 27000) begin
                $display("ERRO: frequencia da combinacao %0d fora de 25%% +- 2%%", n);
                erros = erros + 1;
            end

        if (erros == 0) $display("Simulacao OK (%0d verificacoes)", verificacoes);
        else            $display("Simulacao FALHOU (%0d erros em %0d verificacoes)", erros, verificacoes);
        $finish;
    end

endmodule
```

Listagem 17: Testbench da memória 4x2 (`memoria4x2/memoria4x2_teste1.v`)

```
`timescale 1ns / 1ps

// Testbench da memória 4x2 (fila de 4 posições de 2 bits).
// Parte 1: casos dirigidos (valor inicial, deslocamento de 4 cordas, Shift = 0, clear assíncrono).
// Parte 2: 2000 ciclos com entradas aleatórias comparados com um modelo de referência.
module memoria4x2_teste1();

//   Declaração das entradas
     reg [1:0] Data = 2'b00;
```

```
10      reg Shift = 0, CLR = 0;
11      reg Clk = 0;
12
13  //  Declaração das saídas
14      wire [1:0] Pos0, Pos1, Pos2, Pos3;
15
16  //  Instanciação da UUT e ligação das entradas e saídas
17      Memoria4x2 UUT (
18          .Data(Data),
19          .Shift(Shift),
20          .Clk(Clk),
21          .CLR(CLR),
22          .Pos0(Pos0),
23          .Pos1(Pos1),
24          .Pos2(Pos2),
25          .Pos3(Pos3)
26      );
27
28  //  Relógio de 100 MHz (período de 10ns), como o da Basys3
29      always #5 Clk = ~Clk;
30
31  //  Modelo de referência: fila {Pos3, Pos2, Pos1, Pos0}
32      reg [7:0] referencia = 8'h00;
33      always @(posedge Clk or posedge CLR)
34          if (CLR)        referencia <= 8'h00;
35          else if (Shift) referencia <= {referencia[5:0], Data};
36
37      wire [7:0] fila = {Pos3, Pos2, Pos1, Pos0};
38
39  //  Verificação contínua: a fila tem de ser sempre igual à referência
40      integer erros = 0, verificacoes = 0;
41      always @(fila or referencia) begin
42          #1;
43          verificacoes = verificacoes + 1;
44          if (fila !== referencia) begin
45              if (erros < 10) $display("ERRO t=%0t: fila=%b (esperado %b) Data=%b Shift=%b CLR=%b",
46                                       $time, fila, referencia, Data, Shift, CLR);
47              erros = erros + 1;
48          end
49      end
50
51  //  Verificação direta num instante (para os casos dirigidos)
52      task verifica(input [7:0] esperado, input [8*40:1] caso);
53          begin
54              verificacoes = verificacoes + 1;
55              if (fila !== esperado) begin
56                  $display("ERRO t=%0t (%0s): fila=%b (esperado %b)", $time, caso, fila, esperado);
57                  erros = erros + 1;
58              end
59          end
60      endtask
61
62      integer n;
63      integer semente = 2026;        // semente fixa: resultado repetível
64      initial begin
65          // --- Parte 1: casos dirigidos ------------------------------------------
66          #1   verifica(8'h00, "valor inicial (INIT=0)");
67          #100;                                          // configurações iniciais da FPGA
68          // As entradas mudam a meio do período (relógio sobe em 5, 15, 25, ...)
69          Shift = 1;
70          Data = 2'b01;      #10 verifica(8'b00_00_00_01, "1.a corda em Pos0");
71          Data = 2'b10;      #10 verifica(8'b00_00_01_10, "desloca: Pos1 <- Pos0");
```

```
72          Data = 2'b11;      #10 verifica(8'b00_01_10_11, "3.a corda");
73          Data = 2'b00;      #10 verifica(8'b01_10_11_00, "pausa entra em Pos0");
74          Data = 2'b10;      #10 verifica(8'b10_11_00_10, "Pos3 perde a 1.a corda");
75          Shift = 0; Data = 2'b01;
76                             #50 verifica(8'b10_11_00_10, "Shift=0 mantem 5 ciclos");
77          #2 CLR = 1;        #1  verifica(8'h00, "CLR assincrono (sem flanco)");
78          Shift = 1;         #20 verifica(8'h00, "CLR tem prioridade sobre Shift");
79          CLR = 0; Shift = 0; #7;
80
81          // --- Parte 2: 2000 ciclos com entradas aleatórias ----------------------
82          for (n = 0; n < 2000; n = n + 1) begin
83              Data  = $random(semente);
84              Shift = $random(semente);
85              if (($random(semente) & 63) == 0) begin   // às vezes um clear fora do flanco
86                  #3 CLR = 1;
87                  #4 CLR = 0;
88                  #3;
89              end
90              else
91                  #10;
92          end
93
94          if (erros == 0) $display("Simulacao OK (%0d verificacoes)", verificacoes);
95          else            $display("Simulacao FALHOU (%0d erros em %0d verificacoes)", erros, verificacoes);
96          $finish;
97      end
98
99  endmodule
```

Listagem 18: Testbench do temporizador (`temporizador/temporizador_teste1.v`)

```
1   `timescale 1ns / 1ps
2
3   // Testbench do temporizador.
4   // Verifica que Tempo fica a 1 exatamente Periodo ciclos depois de Start, que volta a 0
5   // sozinho, que Reset interrompe a contagem, que Start durante a contagem recomeça e que
6   // Periodo = 0 não ativa Tempo.
7   module temporizador_teste1();
8
9   //   Declaração das entradas
10      reg Clock = 0;
11      reg Reset = 0, Start = 0;
12      reg [7:0] Periodo = 8'd0;
13
14  //   Declaração da saída
15      wire Tempo;
16
17  //   Instanciação da UUT (contador de 8 bits para a simulação)
18      Temporizador #(.N(8)) UUT (
19          .Clock(Clock),
20          .Reset(Reset),
21          .Start(Start),
22          .Periodo(Periodo),
23          .Tempo(Tempo)
24      );
25
26  //   Relógio de 100 MHz (período de 10ns), como o da Basys3
27      always #5 Clock = ~Clock;
28
29      integer erros = 0, verificacoes = 0;
30      task verifica(input esperado, input [8*48:1] caso);
```

```
31          begin
32              verificacoes = verificacoes + 1;
33              if (Tempo !== esperado) begin
34                  $display("ERRO t=%0t (%0s): Tempo=%b (esperado %b)", $time, caso, Tempo, esperado);
35                  erros = erros + 1;
36              end
37          end
38      endtask
39
40  //   Dá um impulso de Start com 1 ciclo e conta durante quantos ciclos Tempo fica a 1
41      integer ciclos;
42      task temporiza(input [7:0] p);
43          begin
44              Periodo = p; Start = 1;
45              #10 Start = 0;
46              ciclos = 0;
47              while (Tempo === 1'b1 && ciclos < 300) begin
48                  #10 ciclos = ciclos + 1;
49              end
50              verificacoes = verificacoes + 1;
51              if (ciclos != p) begin
52                  $display("ERRO t=%0t: Periodo=%0d mas Tempo esteve a 1 durante %0d ciclos", $time, p,
    ciclos);
53                  erros = erros + 1;
54              end
55          end
56      endtask
57
58      integer n;
59      initial begin
60          #1   verifica(1'b0, "valor inicial: parado");
61          // As entradas mudam a meio do período (relógio sobe em 5, 15, 25, ...)
62          #9;
63          temporiza(8'd1);
64          temporiza(8'd5);
65          temporiza(8'd17);
66          temporiza(8'd255);
67          #50  verifica(1'b0, "fica a 0 depois do fim");
68
69          // Periodo = 0: Tempo nunca sobe
70          Periodo = 0; Start = 1; #10 Start = 0;
71          verifica(1'b0, "Periodo = 0");
72
73          // Reset a meio da contagem
74          Periodo = 50; Start = 1; #10 Start = 0;
75          #100 verifica(1'b1, "a contar antes do Reset");
76          Reset = 1; #10 verifica(1'b0, "Reset interrompe a contagem");
77          Reset = 0; #100 verifica(1'b0, "continua parado depois do Reset");
78
79          // Reset tem prioridade sobre Start
80          Reset = 1; Start = 1; #10 verifica(1'b0, "Reset tem prioridade sobre Start");
81          Reset = 0; Start = 0;
82
83          // Start durante a contagem recomeça com o novo período
84          Periodo = 30; Start = 1; #10 Start = 0;
85          #100;                                   // 10 dos 30 ciclos já passaram
86          temporiza(8'd12);                       // recomeça: tem de durar 12, não 20
87
88          // Vários valores seguidos
89          for (n = 2; n < 40; n = n + 7)
90              temporiza(n);
91
```

```
92          if (erros == 0) $display("Simulacao OK (%0d verificacoes)", verificacoes);
93          else            $display("Simulacao FALHOU (%0d erros em %0d verificacoes)", erros, verificacoes);
94          $finish;
95      end
96
97  endmodule
```

Listagem 19: Testbench do temporizador variável (`temporizador/temporizadorvariavel_teste1.v`)

```
1   `timescale 1ns / 1ps
2
3   // Testbench do temporizador variável (registo da velocidade + temporizador).
4   // Com tempos curtos (INICIAL = 40, PASSO = 6, MINIMO = 20 ciclos) verifica:
5   //   - depois de RstTemp, cada StartTemp dá Tempo = 1 durante INICIAL ciclos;
6   //   - cada Faster encurta o período em PASSO ciclos, sem descer abaixo de MINIMO
7   //     (40, 34, 28, 22 e depois 22 sempre, porque 22 - 6 < 20);
8   //   - RstTemp repõe o período inicial e interrompe a contagem.
9   module temporizadorvariavel_teste1();
10
11  //   Declaração das entradas
12       reg Clock = 0;
13       reg RstTemp = 0, StartTemp = 0, Faster = 0;
14
15  //   Declaração da saída
16       wire Tempo;
17
18       localparam INICIAL = 40, PASSO = 6, MINIMO = 20;
19
20  //   Instanciação da UUT e ligação das entradas e saídas
21       TemporizadorVariavel #(.N(8), .INICIAL(INICIAL), .PASSO(PASSO), .MINIMO(MINIMO)) UUT (
22           .Clock(Clock),
23           .RstTemp(RstTemp),
24           .StartTemp(StartTemp),
25           .Faster(Faster),
26           .Tempo(Tempo)
27       );
28
29  //   Relógio de 100 MHz (período de 10ns), como o da Basys3
30       always #5 Clock = ~Clock;
31
32       integer erros = 0, verificacoes = 0;
33
34  //   Impulso de StartTemp com 1 ciclo; conta os ciclos com Tempo a 1 e compara
35       integer ciclos;
36       task temporiza(input integer esperado);
37           begin
38               StartTemp = 1;
39               #10 StartTemp = 0;
40               ciclos = 0;
41               while (Tempo === 1'b1 && ciclos < 300) begin
42                   #10 ciclos = ciclos + 1;
43               end
44               verificacoes = verificacoes + 1;
45               if (ciclos != esperado) begin
46                   $display("ERRO t=%0t: Tempo esteve a 1 durante %0d ciclos (esperado %0d)", $time, ciclos,
    esperado);
47                   erros = erros + 1;
48               end
49           end
50       endtask
51
```

```
52  //   Impulso de Faster com 1 ciclo (como no estado S5 da máquina de estados)
53       task acelera;
54           begin
55               Faster = 1;
56               #10 Faster = 0;
57           end
58       endtask
59
60       initial begin
61           // As entradas mudam a meio do período (relógio sobe em 5, 15, 25, ...)
62           #10 RstTemp = 1;
63           #10 RstTemp = 0;
64           temporiza(40);
65           temporiza(40);                 // sem Faster o período não muda
66           acelera; temporiza(34);
67           acelera; temporiza(28);
68           acelera; temporiza(22);
69           acelera; temporiza(22);        // 22 - 6 = 16 < MINIMO: fica em 22
70           acelera; temporiza(22);
71
72           // RstTemp a meio da contagem: para e repõe o período inicial
73           StartTemp = 1; #10 StartTemp = 0;
74           #50 RstTemp = 1;
75           #10 RstTemp = 0;
76           verificacoes = verificacoes + 1;
77           if (Tempo !== 1'b0) begin
78               $display("ERRO t=%0t: RstTemp nao parou a contagem", $time);
79               erros = erros + 1;
80           end
81           temporiza(40);
82
83           if (erros == 0) $display("Simulacao OK (%0d verificacoes)", verificacoes);
84           else            $display("Simulacao FALHOU (%0d erros em %0d verificacoes)", erros, verificacoes);
85           $finish;
86       end
87
88   endmodule
```

Listagem 20: Testbench do flip-flop T do LED Beat (`beat/beat_teste1.v`)

```
1   `timescale 1ns / 1ps
2
3   // Testbench do flip-flop T do LED Beat.
4   // Verifica que o LED troca de estado em cada flanco com Tgl a 1, mantém com Tgl a 0
5   // e que CLR o apaga de forma assíncrona.
6   module beat_teste1();
7
8   //   Declaração das entradas
9        reg Clk = 0;
10       reg Tgl = 0, CLR = 0;
11
12  //   Declaração da saída
13       wire Q;
14
15  //   Instanciação da UUT e ligação das entradas e saídas
16       Beat UUT (
17           .Clk(Clk),
18           .Tgl(Tgl),
19           .CLR(CLR),
20           .Q(Q)
21       );
```

```
22
23  //   Relógio de 100 MHz (período de 10ns), como o da Basys3
24       always #5 Clk = ~Clk;
25
26       integer erros = 0, verificacoes = 0;
27       task verifica(input esperado, input [8*40:1] caso);
28           begin
29               verificacoes = verificacoes + 1;
30               if (Q !== esperado) begin
31                   $display("ERRO t=%0t (%0s): Q=%b (esperado %b)", $time, caso, Q, esperado);
32                   erros = erros + 1;
33               end
34           end
35       endtask
36
37       initial begin
38           #1   verifica(1'b0, "valor inicial (INIT=0)");
39           #100;                                          // configurações iniciais da FPGA
40           // As entradas mudam a meio do período (relógio sobe em 5, 15, 25, ...)
41           #9;
42           #30  verifica(1'b0, "Tgl=0 mantem");
43           Tgl = 1;  #10 verifica(1'b1, "1.o Tgl acende");
44           Tgl = 0;  #50 verifica(1'b1, "mantem aceso");
45           Tgl = 1;  #10 verifica(1'b0, "2.o Tgl apaga");
46                     #10 verifica(1'b1, "Tgl continuo: 3.o");
47                     #10 verifica(1'b0, "Tgl continuo: 4.o");
48                     #10 verifica(1'b1, "Tgl continuo: 5.o");
49           Tgl = 0;  #2 CLR = 1;
50                     #1 verifica(1'b0, "CLR assincrono apaga");
51           Tgl = 1;  #20 verifica(1'b0, "CLR tem prioridade sobre Tgl");
52           CLR = 0;  #4 verifica(1'b1, "volta a piscar depois do CLR");
53
54           if (erros == 0) $display("Simulacao OK (%0d verificacoes)", verificacoes);
55           else            $display("Simulacao FALHOU (%0d erros em %0d verificacoes)", erros, verificacoes);
56           $finish;
57       end
58
59  endmodule
```

Listagem 21: Testbench da gestão dos displays (`gestaodisplays/gestaodisplays_teste1.v`)

```
1   `timescale 1ns / 1ps
2
3   // Testbench da gestão dos displays (com o transcodificador das cordas).
4   // Com o varrimento acelerado (METADE = 2), para cada uma das 256 combinações das 4 posições
5   // da memória verifica em cada passo do varrimento que só um display está ativo, que os
6   // segmentos são os da corda que esse display deve mostrar (Pos0 à esquerda, Pos3 à direita)
7   // e que o ponto decimal está apagado. Verifica também que os 4 displays são todos varridos.
8   module gestaodisplays_teste1();
9
10  //   Declaração das entradas
11       reg [1:0] Pos0 = 0, Pos1 = 0, Pos2 = 0, Pos3 = 0;
12       reg Clock = 0;
13
14  //   Declaração das saídas
15       wire [6:0] Segmentos;
16       wire DotPoint;
17       wire [3:0] Anodos;
18
19  //   Instanciação da UUT (varrimento a 100 MHz / 4 para a simulação ser curta)
20       GestaoDisplays #(.METADE(2)) UUT (
```

```
21          .Pos0(Pos0),
22          .Pos1(Pos1),
23          .Pos2(Pos2),
24          .Pos3(Pos3),
25          .Clock(Clock),
26          .Segmentos(Segmentos),
27          .DotPoint(DotPoint),
28          .Anodos(Anodos)
29      );
30
31  //  Relógio de 100 MHz (período de 10ns), como o da Basys3
32      always #5 Clock = ~Clock;
33
34  //  Segmentos esperados para cada corda (abcdefg, ativos a Low)
35      function [6:0] segmentos_de(input [1:0] corda);
36          case (corda)
37              2'b00: segmentos_de = 7'b1111111;   // pausa
38              2'b01: segmentos_de = 7'b0111111;   // cima (a)
39              2'b10: segmentos_de = 7'b1111110;   // meio (g)
40              2'b11: segmentos_de = 7'b1110111;   // baixo (d)
41          endcase
42      endfunction
43
44      integer erros = 0, verificacoes = 0;
45      reg [3:0] varridos;            // displays que já estiveram ativos
46      reg [1:0] corda;
47
48      task verifica_passo;
49          begin
50              verificacoes = verificacoes + 1;
51              case (Anodos)
52                  4'b0111: corda = Pos0;     // display da esquerda
53                  4'b1011: corda = Pos1;
54                  4'b1101: corda = Pos2;
55                  4'b1110: corda = Pos3;     // display da direita
56                  default: begin
57                      $display("ERRO t=%0t: Anodos=%b (deve haver um so display ativo)", $time, Anodos);
58                      erros = erros + 1;
59                  end
60              endcase
61              varridos = varridos | ~Anodos;
62              if (Segmentos !== segmentos_de(corda) || DotPoint !== 1'b1) begin
63                  if (erros < 10) $display("ERRO t=%0t: Anodos=%b Pos=%b %b %b %b Segmentos=%b (esperado %b)
    DP=%b",
64                                           $time, Anodos, Pos0, Pos1, Pos2, Pos3, Segmentos,
    segmentos_de(corda), DotPoint);
65                  erros = erros + 1;
66              end
67          end
68      endtask
69
70      integer n, p;
71      initial begin
72          #100;                                          // configurações iniciais da FPGA
73          for (n = 0; n < 256; n = n + 1) begin
74              {Pos0, Pos1, Pos2, Pos3} = n;
75              varridos = 4'b0000;
76              // o display ativo muda a cada 40 ns (METADE = 2): verifica 8 passos fora dos flancos do
    relógio
77              #22;
78              for (p = 0; p < 8; p = p + 1) begin
79                  verifica_passo;
```

```
                #40;
            end
            #18;
            verificacoes = verificacoes + 1;
            if (varridos !== 4'b1111) begin
                $display("ERRO t=%0t: nem todos os displays foram varridos (%b)", $time, varridos);
                erros = erros + 1;
            end
        end

        if (erros == 0) $display("Simulacao OK (%0d verificacoes)", verificacoes);
        else            $display("Simulacao FALHOU (%0d erros em %0d verificacoes)", erros, verificacoes);
        $finish;
    end

endmodule
```

Listagem 22: Testbench do controlador (`controlador/controlador_teste1.v`)

```
`timescale 1ns / 1ps

// Testbench do controlador (máquina de estados por realização direta).
// Um modelo de referência do fluxograma (descrição comportamental com case) corre em paralelo
// e, a meio de cada ciclo e logo depois de cada Reset, compara-se o estado (os 6 flip-flops) e
// todas as saídas. Verifica-se também que há sempre um e um só flip-flop ativo.
// Parte 1: percurso dirigido por todos os estados e transições do fluxograma.
// Parte 2: 5000 ciclos com Start e Tempo aleatórios e Resets assíncronos fora do flanco.
module controlador_teste1();

//   Declaração das entradas
    reg Clock = 0;
    reg Reset = 0, Start = 0, Tempo = 0;

//   Declaração das saídas
    wire ClrMem, InitLFSR, EnLFSR, ShiftMem, RstTemp, StartTemp, Faster, TglBeat, Ready, Waiting;
    wire [5:0] Estado;

//   Instanciação da UUT e ligação das entradas e saídas
    Controlador UUT (
        .Clock(Clock),
        .Reset(Reset),
        .Start(Start),
        .Tempo(Tempo),
        .ClrMem(ClrMem),
        .InitLFSR(InitLFSR),
        .EnLFSR(EnLFSR),
        .ShiftMem(ShiftMem),
        .RstTemp(RstTemp),
        .StartTemp(StartTemp),
        .Faster(Faster),
        .TglBeat(TglBeat),
        .Ready(Ready),
        .Waiting(Waiting),
        .Estado(Estado)
    );

//   Relógio de 100 MHz (período de 10ns), como o da Basys3
    always #5 Clock = ~Clock;

//   Modelo de referência: o fluxograma descrito de forma comportamental
    integer ref_estado = 0;              // número do estado (0 a 5)
```

```
43      always @(posedge Clock or posedge Reset)
44          if (Reset) ref_estado <= 0;
45          else
46              case (ref_estado)
47                  0: ref_estado <= 1;
48                  1: ref_estado <= Start ? 2 : 1;
49                  2: ref_estado <= Start ? 2 : 3;
50                  3: ref_estado <= 4;
51                  4: ref_estado <= Tempo ? 4 : 5;
52                  5: ref_estado <= 3;
53              endcase
54
55  //   Saídas esperadas em cada estado:  ClrMem InitLFSR EnLFSR ShiftMem RstTemp StartTemp Faster TglBeat
    Ready Waiting
56      function [9:0] saidas_de(input integer e);
57          case (e)
58              0: saidas_de = 10'b1_1_1_0_1_0_0_0_0_0;
59              1: saidas_de = 10'b0_0_1_0_0_0_0_0_1_0;
60              2: saidas_de = 10'b0_0_1_0_0_0_0_0_1_0;
61              3: saidas_de = 10'b0_0_1_1_0_1_0_1_0_0;
62              4: saidas_de = 10'b0_0_1_0_0_0_0_0_0_1;
63              5: saidas_de = 10'b0_0_1_0_0_0_1_0_0_0;
64              default: saidas_de = 10'bx;
65          endcase
66      endfunction
67
68      wire [9:0] saidas = {ClrMem, InitLFSR, EnLFSR, ShiftMem, RstTemp, StartTemp, Faster, TglBeat, Ready,
    Waiting};
69
70      integer erros = 0, verificacoes = 0;
71      integer visitas [0:5];         // quantas vezes se verificou cada estado
72      task verifica(input [8*40:1] caso);
73          begin
74              verificacoes = verificacoes + 1;
75              visitas[ref_estado] = visitas[ref_estado] + 1;
76              if (Estado !== (6'b000001 << ref_estado) || saidas !== saidas_de(ref_estado)) begin
77                  if (erros < 10) $display("ERRO t=%0t (%0s): Estado=%b saidas=%b (esperado S%0d: %b %b)
    Start=%b Tempo=%b",
78                                           $time, caso, Estado, saidas, ref_estado, 6'b000001 << ref_estado,
    saidas_de(ref_estado), Start, Tempo);
79                  erros = erros + 1;
80              end
81          end
82      endtask
83
84  //   Verificação a meio de cada ciclo (flanco descendente), depois das configurações iniciais
85      always @(negedge Clock)
86          if ($time > 100) verifica("a meio do ciclo");
87
88  //   Um ciclo de relógio; as entradas mudam 2 ns depois do flanco ascendente
89      task ciclo(input s, input t);
90          begin
91              Start = s; Tempo = t;
92              #10;
93          end
94      endtask
95
96      integer n;
97      integer semente = 2026;        // semente fixa: resultado repetível
98      initial begin
99          visitas[0] = 0; visitas[1] = 0; visitas[2] = 0; visitas[3] = 0; visitas[4] = 0; visitas[5] = 0;
100         #1   verifica("estado inicial: S0 (INIT)");
```

```
        #106;                                           // configurações iniciais (t = 107)

        // --- Parte 1: percurso dirigido ----------------------------------------
        Reset = 1;   #1 verifica("Reset assincrono: S0");
        #20 Reset = 0;                                  // t = 128
        #9;                                             // t = 137: S1
        ciclo(0, 0); ciclo(0, 0); ciclo(0, 1);          // S1 espera Start (Tempo nao interessa)
        ciclo(1, 0);                                    // Start = 1: S2
        ciclo(1, 0); ciclo(1, 1);                       // S2 enquanto Start = 1
        ciclo(0, 0);                                    // Start = 0: S3
        ciclo(0, 1);                                    // S3 -> S4 incondicional
        ciclo(0, 1); ciclo(1, 1); ciclo(0, 1);          // S4 enquanto Tempo = 1 (Start nao interessa)
        ciclo(0, 0);                                    // Tempo = 0: S5
        ciclo(1, 1);                                    // S5 -> S3 incondicional
        ciclo(0, 1); ciclo(0, 1); ciclo(0, 0);          // S3 -> S4 -> S5 ...
        ciclo(0, 0); ciclo(0, 0);
        #4 Reset = 1;  #1 verifica("Reset a meio do jogo");
        #12 Reset = 0;
        #3;

        // --- Parte 2: entradas aleatórias e Resets fora do flanco ---------------
        for (n = 0; n < 5000; n = n + 1) begin
            // Start muda poucas vezes (tecla), Tempo fica a 1 durante vários ciclos
            if (($random(semente) & 7) == 0)  Start = ~Start;
            if (($random(semente) & 3) == 0)  Tempo = ~Tempo;
            if (($random(semente) & 255) == 0) begin
                #4 Reset = 1;
                #1 verifica("Reset aleatorio");
                #(10 * (1 + ($random(semente) & 3))) Reset = 0;
                #5;
            end
            else
                #10;
        end

        for (n = 0; n < 6; n = n + 1)
            if (visitas[n] == 0) begin
                $display("ERRO: o estado S%0d nunca foi visitado", n);
                erros = erros + 1;
            end
        $display("Ciclos verificados por estado: S0 %0d, S1 %0d, S2 %0d, S3 %0d, S4 %0d, S5 %0d",
                 visitas[0], visitas[1], visitas[2], visitas[3], visitas[4], visitas[5]);
        if (erros == 0) $display("Simulacao OK (%0d verificacoes)", verificacoes);
        else            $display("Simulacao FALHOU (%0d erros em %0d verificacoes)", erros, verificacoes);
        $finish;
    end

endmodule
```

Listagem 23: Testbench do sistema completo (`guitarhero/guitarhero_teste1.v`)

```
`timescale 1ns / 1ps

// Testbench do circuito completo (controlador + arquitetura a controlar).
// Tempos curtos: varrimento METADE = 2, anti-ressalto de 4 ciclos e temporizador com
// INICIAL = 40, PASSO = 4, MINIMO = 20 ciclos.
// Em cada ciclo verifica-se:
//   - há um e um só estado ativo e os LEDs Ready e Waiting correspondem ao estado;
//   - a memória é igual a uma fila de referência, alimentada por um modelo do LFSR do próprio
//     testbench e comandada só pelos LEDs dos estados (S0 limpa, S3 desloca);
//   - o display ativo mostra a corda da posição certa da memória;
```

```
11 //   - o LED Beat troca de estado uma vez por passo;
12 //   - o intervalo entre passos é o período do temporizador (40, 36, 32, ... até 20) + 3 ciclos
13 //     (estados S4 com Tempo = 0, S5 e S3).
14 // O percurso: Reset (ativação imediata, libertação síncrona), espera em S1, Start com ressaltos,
15 // 12 passos do jogo, Reset a meio do jogo e novo jogo.
16 module guitarhero_teste1();
17
18 //   Declaração das entradas
19      reg CLKBasys = 0;
20      reg BtnReset = 0, BtnStart = 0;
21
22 //   Declaração das saídas
23      wire [6:0] Seg;
24      wire DP;
25      wire [3:0] An;
26      wire LedReady, LedBeat, LedWaiting;
27      wire [5:0] LedEstado;
28
29      localparam INICIAL = 40, PASSO = 4, MINIMO = 20;
30
31 //   Instanciação da UUT com tempos curtos para a simulação
32      GuitarHero #(.METADE(2), .ESTAVEL(4), .INICIAL(INICIAL), .PASSO(PASSO), .MINIMO(MINIMO)) UUT (
33          .CLKBasys(CLKBasys),
34          .BtnReset(BtnReset),
35          .BtnStart(BtnStart),
36          .Seg(Seg),
37          .DP(DP),
38          .An(An),
39          .LedReady(LedReady),
40          .LedBeat(LedBeat),
41          .LedWaiting(LedWaiting),
42          .LedEstado(LedEstado)
43      );
44
45 //   Relógio de 100 MHz (período de 10ns), como o da Basys3
46      always #5 CLKBasys = ~CLKBasys;
47
48      integer erros = 0, verificacoes = 0;
49      task erro(input [8*60:1] texto);
50          begin
51              if (erros < 15) $display("ERRO t=%0t: %0s (Estado=%b)", $time, texto, LedEstado);
52              erros = erros + 1;
53          end
54      endtask
55
56 //   Segmentos esperados para cada corda (abcdefg, ativos a Low)
57      function [6:0] segmentos_de(input [1:0] corda);
58          case (corda)
59              2'b00: segmentos_de = 7'b1111111;
60              2'b01: segmentos_de = 7'b0111111;
61              2'b10: segmentos_de = 7'b1111110;
62              2'b11: segmentos_de = 7'b1110111;
63          endcase
64      endfunction
65
66 //   Modelo do LFSR (independente da UUT): carrega a semente em S0 e avança em todos os estados
67      localparam [31:0] SEMENTE = 32'h03E3180A;
68      reg [31:0] lfsr_ref = 32'h0;
69      wire [1:0] corda_ref = {lfsr_ref[31], lfsr_ref[0]};
70      always @(posedge CLKBasys)
71          if (LedEstado[0]) lfsr_ref <= SEMENTE;
72          else              lfsr_ref <= {lfsr_ref[30:0], ~(lfsr_ref[31] ^ lfsr_ref[21] ^ lfsr_ref[1] ^
```

```
    lfsr_ref[0])};
73
74  //   Fila de referência {Pos3, Pos2, Pos1, Pos0}: limpa em S0, recebe a corda do modelo em S3
75       reg [7:0] fila_ref = 8'h00;
76       reg [3:0] cordas_vistas = 4'b0000;
77       always @(posedge CLKBasys or posedge LedEstado[0])
78           if (LedEstado[0]) fila_ref <= 8'h00;
79           else if (LedEstado[3]) begin
80               fila_ref <= {fila_ref[5:0], corda_ref};
81               cordas_vistas[corda_ref] <= 1'b1;
82           end
83
84       wire [7:0] fila = {UUT.Pos3, UUT.Pos2, UUT.Pos1, UUT.Pos0};
85
86  //   Verificações a meio de cada ciclo
87       integer ciclo = 0, ultimo_passo = -1, passos = 0, periodo = INICIAL;
88       reg [1:0] corda;
89       always @(negedge CLKBasys)
90           if ($time > 100) begin
91               ciclo = ciclo + 1;
92               verificacoes = verificacoes + 1;
93               if (LedEstado[0]) begin                      // S0: tudo volta ao início
94                   ultimo_passo = -1; passos = 0; periodo = INICIAL;
95               end
96               // um e um só estado ativo
97               case (LedEstado)
98                   6'b000001, 6'b000010, 6'b000100, 6'b001000, 6'b010000, 6'b100000: ;
99                   default: erro("estado nao one-hot");
100              endcase
101              // LEDs de sinalização
102              if (LedReady !== (LedEstado[1] | LedEstado[2])) erro("LED Ready errado");
103              if (LedWaiting !== LedEstado[4])                erro("LED Waiting errado");
104              if (LedBeat !== passos[0])                      erro("LED Beat errado");
105              // memória e displays
106              if (fila !== fila_ref) erro("memoria diferente da fila de referencia");
107              case (An)
108                  4'b0111: corda = UUT.Pos0;
109                  4'b1011: corda = UUT.Pos1;
110                  4'b1101: corda = UUT.Pos2;
111                  4'b1110: corda = UUT.Pos3;
112                  default: erro("mais do que um display ativo");
113              endcase
114              if (Seg !== segmentos_de(corda) || DP !== 1'b1) erro("segmentos errados no display ativo");
115              // passo do jogo (estado S3): intervalo desde o passo anterior
116              if (LedEstado[3]) begin
117                  if (ultimo_passo >= 0) begin
118                      if (ciclo - ultimo_passo != periodo + 3) begin
119                          $display("ERRO t=%0t: passo %0d ao fim de %0d ciclos (esperado %0d)",
120                                   $time, passos, ciclo - ultimo_passo, periodo + 3);
121                          erros = erros + 1;
122                      end
123                      if (periodo >= MINIMO + PASSO) periodo = periodo - PASSO;
124                  end
125                  ultimo_passo = ciclo;
126                  passos = passos + 1;
127              end
128          end
129
130  //   Verificação do estado num instante
131       task verifica_estado(input [5:0] esperado, input [8*40:1] caso);
132           begin
133               verificacoes = verificacoes + 1;
```

```
134             if (LedEstado !== esperado) begin
135                 $display("ERRO t=%0t (%0s): Estado=%b (esperado %b)", $time, caso, LedEstado, esperado);
136                 erros = erros + 1;
137             end
138         end
139     endtask
140
141 //  Inicialização assíncrona: logo depois de carregar Reset (sem esperar pelo relógio)
142     task verifica_reset;
143         begin
144             #1 verifica_estado(6'b000001, "Reset: S0 imediatamente");
145             verificacoes = verificacoes + 1;
146             if (fila !== 8'h00 || LedBeat !== 1'b0 || LedReady !== 1'b0 || LedWaiting !== 1'b0) begin
147                 $display("ERRO t=%0t: Reset nao limpou memoria/LEDs (fila=%b Beat=%b Ready=%b Waiting=%b)",
148                          $time, fila, LedBeat, LedReady, LedWaiting);
149                 erros = erros + 1;
150             end
151         end
152     endtask
153
154 //  Tecla Start com ressaltos (mais curtos do que o anti-ressalto)
155     task carrega_start;
156         begin
157             BtnStart = 1; #13 BtnStart = 0; #7 BtnStart = 1; #300;
158         end
159     endtask
160     task larga_start;
161         begin
162             BtnStart = 0; #9 BtnStart = 1; #5 BtnStart = 0;
163         end
164     endtask
165
166     initial begin
167         #100;                                           // configurações iniciais da FPGA
168         #33 BtnReset = 1;                               // t = 133
169         verifica_reset;
170         #98 BtnReset = 0;                               // t = 232
171         // a libertação passa pelo sincronizador (2 flancos): S0 até ao 3.º flanco
172         @(negedge CLKBasys); @(negedge CLKBasys);
173         verifica_estado(6'b000001, "S0 durante a libertacao sincrona");
174         @(negedge CLKBasys);
175         verifica_estado(6'b000010, "S1 depois do Reset");
176         verificacoes = verificacoes + 1;
177         if (UUT.lfsr.regLFSR !== 32'h03E3180A) begin
178             $display("ERRO t=%0t: LFSR sem a semente depois de S0 (%h)", $time, UUT.lfsr.regLFSR);
179             erros = erros + 1;
180         end
181         #500 verifica_estado(6'b000010, "espera em S1 sem Start");
182
183         carrega_start;
184         verifica_estado(6'b000100, "Start carregada: S2 (sem sair com os ressaltos)");
185         #200 verifica_estado(6'b000100, "S2 enquanto Start carregada");
186         larga_start;
187         #100;
188         if (LedReady !== 1'b0 || LedWaiting !== 1'b1) erro("depois de largar Start: Ready apagado e Waiting
    aceso");
189
190         wait (passos == 12);
191         verificacoes = verificacoes + 1;
192         if (cordas_vistas != 4'b1111) begin
193             $display("ERRO: em 12 passos so sairam as cordas %b", cordas_vistas);
194             erros = erros + 1;
```

```
195          end
196
197          // Reset a meio do jogo, fora do flanco do relógio
198          #37 BtnReset = 1;
199          verifica_reset;
200          #48 BtnReset = 0;
201          #300 verifica_estado(6'b000010, "S1 depois do Reset a meio do jogo");
202          if (UUT.Tempo !== 1'b0) erro("temporizador nao foi inicializado");
203
204          // Novo jogo: o período volta ao inicial (verificado a cada passo)
205          carrega_start;
206          larga_start;
207          wait (passos == 4);
208          #50;
209
210          if (erros == 0) $display("Simulacao OK (%0d verificacoes)", verificacoes);
211          else            $display("Simulacao FALHOU (%0d erros em %0d verificacoes)", erros, verificacoes);
212          $finish;
213      end
214
215      initial begin
216          #100000 $display("Simulacao FALHOU (tempo limite: o jogo nao avancou)");
217          $finish;
218      end
219
220  endmodule
```